

2026年江南大学 第十届校内选拔赛

参赛论文

题目：我国碳排放时空特征分析与趋势预测

学院：光电信息物理与科学学院

学号：1130524220 姓名：胡珩瑄

学号：1130524218 姓名：王觉幸

学号：1130524217 姓名：陈平原

2026 年 5 月 20 日

我国碳排放时空特征分析与趋势预测

摘要

为把握我国碳排放空间差异、识别关键驱动因素并研判中长期达峰趋势，本文基于 2019–2025 年全国碳排放日度数据、2022 年 30 省碳排放清单及相关统计数据，构建“空间差异评价–省份分类分级–驱动因素识别–多情景预测”的综合建模框架。首先，选取碳排放总量、人均碳排放、碳排放强度和煤炭相关排放占比等指标，利用变异系数、基尼系数、泰尔指数和省级空间热力图刻画区域差异，并结合熵权 TOPSIS 模型评价各省低碳发展水平。其次，采用 K-means 聚类将省份划分为资源依赖高排放型、工业制造高排放型、中等转型压力型和经济发达效率型。再次，构建 STIRPAT 模型识别碳排放驱动因素，并以岭回归和交叉验证检验稳健性。最后，以 2024 年全国排放为基准，利用 STIRPAT 系数递推预测 2026–2045 年基准、低碳和强化低碳三种情景下的碳排放总量与强度。结果表明，2022 年省际碳排放呈现“北方资源能源高排放带、东部沿海制造业高排放带、中西南相对低排放区”的空间格局；30 省碳排放总量基尼系数为 0.3549，碳排放强度基尼系数为 0.3883，排放效率差异更突出。STIRPAT 估计显示，人口规模和煤炭相关排放占比是最稳定的正向驱动因素，模型调整 R^2 为 0.8504。情景预测显示，基准情景下全国碳排放约在 2032 年达到峰值 12435 Mt；低碳情景约在 2026 年达峰；强化低碳情景下降幅最大，2045 年排放降至约 7114 Mt。本文据此提出分区域、分行业、分阶段推进低碳转型的政策建议。

关键词：碳排放；空间差异；熵权 TOPSIS；K-means 聚类；STIRPAT 模型；情景预测

1 问题重述

实现碳达峰、碳中和是我国推动高质量发展和经济社会绿色转型的重要战略目标。我国幅员辽阔，不同省份在资源禀赋、人口规模、产业结构、能源结构和经济发展水平方面差异显著，由此导致碳排放在空间分布、排放效率和减排压力上呈现较强不均衡性。准确刻画省际碳排放差异，识别排放增长的主要驱动因素，并预测未来排放走势，对制定差异化、可操作的减排政策具有重要意义。

题目要求结合附件碳排放数据及相关统计资料，完成以下四项任务。第一，选取碳排放总量、人均碳排放、碳排放强度等核心指标，分析各省碳排放空间分布特征与差异显著性，并综合排放规模、排放效率和经济关联度建立分类分级模型。第二，选取能源消费结构、产业结构、人均 GDP、城镇化率、技术水平等关键指标，识别影响碳排放的核心驱动因素，构建定量预测模型并完成参数估计、显著性检验与模型优化。第三，基于最优预测模型设置基准、低碳、强化低碳三种情景，预测 2026–2045 年全国碳排放总量与强度变化趋势，判断碳达峰时间节点、峰值水平和减排潜力。第四，结合前三问结果，从能源结构优化、产业升级、区域差异化减排、技术创新和政策机制完善等方面提出针对性政策建议。

2 模型假设及符号说明

2.1 模型假设

为保证模型建立和计算过程具有可操作性，本文作如下假设：

- (1) 附件中碳排放数据真实可靠，各省统计口径在同一年份内具有可比性。
- (2) 2022 年省级横截面数据能够反映当前我国省际碳排放差异和主要结构特征。
- (3) 人口、经济发展水平、能源结构、产业结构和城镇化水平是影响碳排放的重要因素，且可通过 STIRPAT 模型进行定量刻画。
- (4) 全国中长期碳排放趋势可由人口、人均经济水平、煤炭占比、产业结构和城镇化率共同决定，且这些因素对碳排放的影响可由 STIRPAT 系数刻画。
- (5) 情景预测中，各驱动变量变化率在 2026–2045 年间平滑变化，不考虑突发重大外部冲击。
- (6) 附件 1 中 2025 年全国碳排放数据截至 2025 年 9 月 30 日，不能直接作为完整年度数据与 2019–2024 年进行同比比较，预测基准采用 2024 年全国排放量。

2.2 符号说明

表 1 主要符号说明

符号	含义
C_i	第 i 个省份碳排放总量
P_i	第 i 个省份人口规模
GDP_i	第 i 个省份地区生产总值
PC_i	第 i 个省份人均碳排放
CI_i	第 i 个省份碳排放强度
$Coal_i$	第 i 个省份煤炭相关排放占比
IS_i	第 i 个省份第二产业占比
A_i	第 i 个省份人均 GDP
U_i	第 i 个省份城镇化率
z_{ij}	第 i 个省份第 j 个指标标准化值
w_j	第 j 个评价指标权重
S_i	第 i 个省份 TOPSIS 综合贴近度
G	基尼系数
T	泰尔指数
CV	变异系数
g_P	人口增长率
g_A	人均 GDP 增长率
$\Delta Coal$	煤炭相关排放占比变化量
ΔIS	第二产业占比变化量
ΔU	城镇化率变化量

3 问题分析及模型

3.1 问题一分析及模型

问题一要求分析省际碳排放空间差异，并对各省碳排放水平进行分类分级。单独使用碳排放总量容易忽视人口规模、经济发展水平和产业结构差异，因此本文从排放规模、排放效率、能源结构、产业结构和经济发展水平五个方面构建指标体系。

首先，采用变异系数、基尼系数和泰尔指数衡量各省碳排放差异。变异系数反映相对离散程度，基尼系数反映空间分布不均衡程度，泰尔指数可进一步衡量整体差异强弱。其计算公式为

$$CV = \frac{\sigma}{\mu},$$
$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n},$$
$$T = \frac{1}{n} \sum_{i=1}^n \frac{x_i}{\bar{x}} \ln \left(\frac{x_i}{\bar{x}} \right).$$

同时，为更直观刻画省际排放的地理集聚特征，本文将 2022 年各省 CO₂ 排放总量与省级行政边界匹配，绘制全国省级碳排放空间分布热力图。热力图可直接反映高排放

省份在华北、内蒙古及东部沿海工业带附近的空间集中现象，从而弥补单纯排序图无法体现地理位置关系的不足。

其次，构建熵权 TOPSIS 综合评价模型。TOPSIS 方法可通过理想解与负理想解距离刻画评价对象的相对优劣，本文结合熵权法确定客观权重后进行综合评价 [1]。对正向指标和负向指标分别标准化：

$$z_{ij} = \frac{x_{ij} - \min x_j}{\max x_j - \min x_j}, \quad z_{ij} = \frac{\max x_j - x_{ij}}{\max x_j - \min x_j}.$$

计算指标比重、熵值和权重：

$$p_{ij} = \frac{z_{ij}}{\sum_{i=1}^n z_{ij}}, \quad e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij}, \quad w_j = \frac{1 - e_j}{\sum_{j=1}^m (1 - e_j)}.$$

最后计算 TOPSIS 贴近度：

$$S_i = \frac{D_i^-}{D_i^+ + D_i^-}.$$

S_i 越大，说明该省低碳发展综合表现越好。

最后，采用 K-means 聚类对省份进行分类。K-means 聚类通过最小化类内平方误差将样本划分为若干类别，适合对多指标省份样本进行类型识别 [2]。输入变量为

$$X_i = (C_i, PC_i, CI_i, Coal_i, IS_i, A_i, U_i),$$

聚类目标函数为

$$\min \sum_{k=1}^K \sum_{i \in G_k} \|X_i - \mu_k\|^2.$$

3.2 问题二分析及模型

问题二要求识别影响碳排放的关键因素并构建预测模型。碳排放不仅受人口规模和经济发展影响，也与能源结构、产业结构和城镇化水平有关。为兼顾解释性和可估计性，本文采用 STIRPAT 模型进行驱动因素识别 [3]。

STIRPAT 模型由 IPAT 恒等式扩展而来：

$$I = aP^bA^cT^de.$$

结合本文数据，将其扩展为

$$\ln C_i = \alpha + \beta_1 \ln P_i + \beta_2 \ln A_i + \beta_3 \ln Coal_i + \beta_4 \ln IS_i + \beta_5 \ln U_i + \varepsilon_i.$$

其中， $\ln P_i$ 表示人口规模， $\ln A_i$ 表示人均 GDP， $\ln Coal_i$ 表示煤炭相关排放占比， $\ln IS_i$ 表示第二产业占比， $\ln U_i$ 表示城镇化率。

为检验模型可靠性，本文采用普通最小二乘法估计参数，并通过 R^2 、调整 R^2 、F 检验、t 检验评价拟合效果和变量显著性。同时采用 VIF 检验多重共线性，并使用岭回归进行稳健性检验。岭回归目标函数为

$$\min \left\{ \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}.$$

3.3 问题三分析及模型

问题三要求预测 2026–2045 年全国碳排放趋势，并判断达峰年份和减排潜力。由于全国年度样本较短，若直接建立多变量时间序列回归，参数稳定性不足。因此本文以问题二估计得到的 STIRPAT 系数为核心，构建锚定基准年的情景递推预测模型。

问题二得到的 STIRPAT 模型为

$$\ln C = \alpha + \beta_1 \ln P + \beta_2 \ln A + \beta_3 Coal + \beta_4 IS + \beta_5 U + \varepsilon.$$

将其转化为年度相对变化形式：

$$\Delta \ln C_t = \beta_1 \Delta \ln P_t + \beta_2 \Delta \ln A_t + \beta_3 \Delta Coal_t + \beta_4 \Delta IS_t + \beta_5 \Delta U_t.$$

于是年度递推公式为

$$C_t = C_{t-1} \exp(\Delta \ln C_t).$$

其中 P 表示人口规模， A 表示人均 GDP， $Coal$ 表示煤炭相关排放占比， IS 表示第二产业占比， U 表示城镇化率。该处理方式既保留 STIRPAT 模型的驱动因素解释，又避免直接使用省级横截面模型截距预测全国绝对排放量；全国排放水平由 2024 年实际排放锚定。

本文设置三类情景：基准情景表示人均 GDP 保持中速增长、煤炭占比和第二产业占比缓慢下降；低碳情景表示煤炭占比下降和产业结构优化速度加快；强化低碳情景表示煤炭替代、产业升级和低碳城镇化进一步强化。人口增长率和人均 GDP 增长率为年度相对增长率；煤炭占比、第二产业占比和城镇化率为年度占比变化量，单位为百分点。达峰判断标准为：在预测区间内寻找碳排放总量最大值，其对应年份即为达峰年份，最大值即为峰值水平。

为检验情景预测结论的稳健性，本文进一步设置两类灵敏度分析。第一，对人口增长率、人均 GDP 增长率、煤炭占比下降速度、第二产业占比下降速度和城镇化率年变化分别进行 $\pm 10\%$ 、 $\pm 20\%$ 单因素扰动，观察达峰年份、峰值水平和 2045 年排放变化。其中“下降速度 +20%”表示负向占比变化的绝对幅度增大，即占比下降更快。第二，对 STIRPAT 模型中的核心系数分别进行 $\pm 10\%$ 扰动，检验问题二参数估计误差对问题三预测结论的影响。

3.4 问题四分析

问题四要求提出政策建议。政策建议应直接服务于前三问结论：问题一提供省份类型，问题二揭示关键驱动因素，问题三给出未来趋势和达峰压力。因此，本文从区域分类、能源结构、产业结构、技术创新和政策机制五个角度提出建议。

4 求解方法、结果及其分析和检验

4.1 问题一求解结果及分析

基于 2022 年 30 省数据，首先计算碳排放总量、人均碳排放、碳排放强度和煤炭相关排放占比的空间差异指标，结果如表2所示。

表 2 省际碳排放空间差异指标

指标	均值	变异系数	基尼系数	泰尔指数	最小值	最大值
CO ₂ 总量/Mt	380.2822	0.6458	0.3549	0.2041	44.6919	920.9053
人均 CO ₂ /吨每人	9.8017	0.7957	0.3513	0.2312	3.2763	36.6002
碳排放强度/吨每万元 GDP	1.3011	0.7958	0.3883	0.2555	0.1720	4.8805
煤炭相关排放占比	0.7113	0.2510	0.1263	0.0447	0.0212	0.9399

结合图1可以看出，2022 年我国 30 省碳排放并非均匀分布，而是呈现明显的地理集聚特征。高排放区域首先集中在华北-环渤海及内蒙古能源基地一带，山东、内蒙古、河北、山西、辽宁构成北方高排放集聚区。其中山东、河北、辽宁主要与钢铁、石化、装备制造等重工业基础有关，内蒙古、山西则更多体现为煤炭资源开发和能源外送带来的高排放特征。其次，东部沿海制造业省份排放总量也较高，江苏、广东、浙江、福建等省沿海成带分布，说明经济规模、工业制造和外向型产业集聚对排放总量具有显著抬升作用。再次，中西部和西南多数省份排放总量处于中低水平，但新疆因能源资源开发形成西北地区局部高值；青海、海南、北京等省市排放总量较低，与人口规模、产业结构和生态功能定位有关。因此，省际碳排放空间格局可概括为“北方资源能源高排放带、东部沿海制造业高排放带、中西南相对低排放区、局部资源型高值点”。

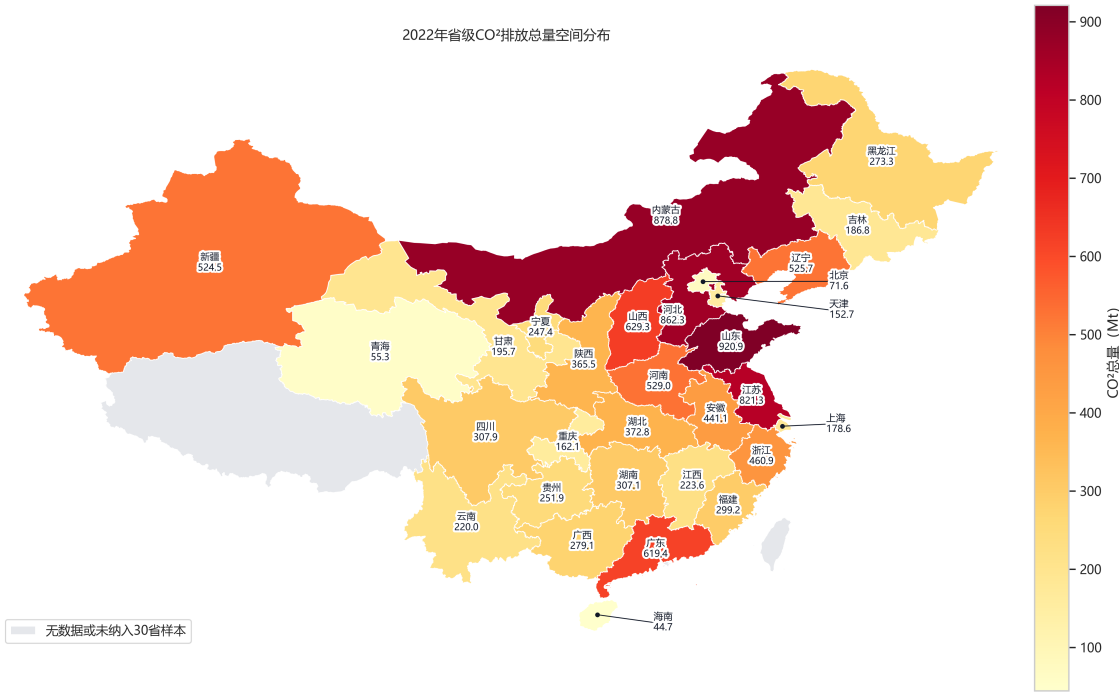


图 1 2022 年省级 CO₂ 排放总量空间分布热力图

差异显著性方面，2022 年 30 省 CO₂ 总量均值为 380.2822 Mt，但最大值山东为 920.9053 Mt，最小值海南为 44.6919 Mt，二者相差约 20.61 倍；排放总量前 8 位省份合计占 30 省总排放的 50.72%，说明排放明显集中在少数重点省份。统计指标进一步验证了这种空间不均衡：CO₂ 总量变异系数为 0.6458、基尼系数为 0.3549、泰尔指数为 0.2041，表明省际排放总量差异显著。人均 CO₂ 变异系数为 0.7957，碳排放强度变异系数为 0.7958、基尼系数为 0.3883，说明资源型地区和重工业地区不仅排放总量高，单位人口和单位经济产出的排放压力也更突出。

熵权 TOPSIS 评价结果显示，指标权重前三位为人均 GDP、煤炭相关排放占比和城镇化率，权重分别为 0.2728、0.2215 和 0.1717。这说明综合低碳发展水平不仅取决于排放本身，还与经济发展质量、能源结构和城市治理能力密切相关。TOPSIS 得分排名靠前的省份为北京、上海、天津、海南、浙江、福建、重庆、广东；排名靠后的省份为山西、内蒙古、新疆、河北、宁夏、河南、山东、甘肃。前者通常具有较低排放强度、较高人均 GDP 或较优能源结构，后者则主要受到煤炭占比高、排放强度高或工业结构偏重的影响。

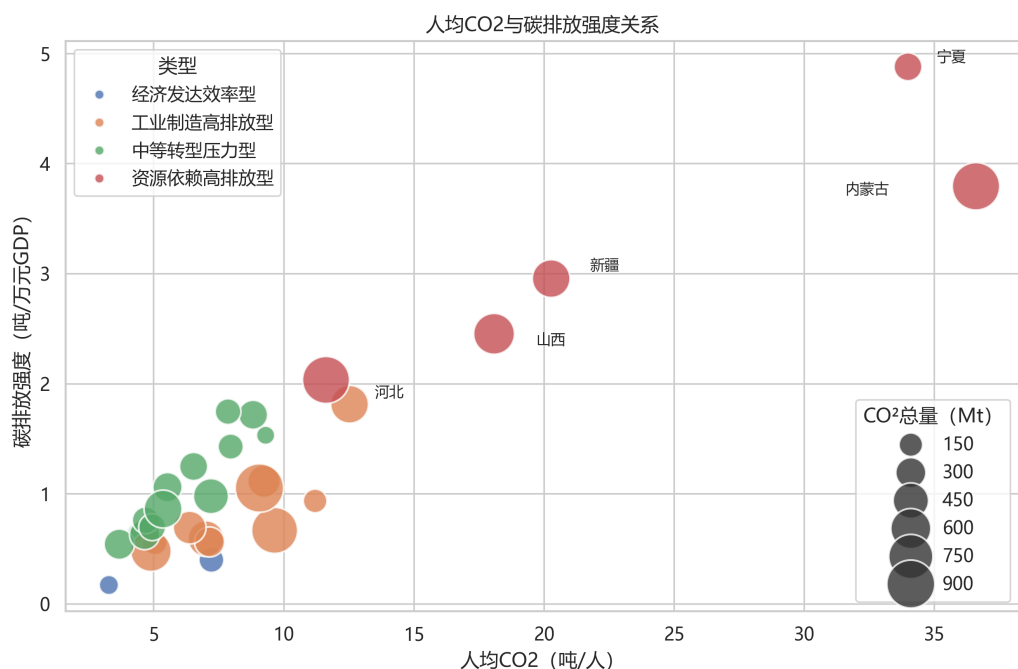


图 2 人均 CO₂ 与碳排放强度关系

聚类模型检验中，K=2 至 6 的轮廓系数分别为 0.349、0.405、0.284、0.282 和 0.312。K=3 时轮廓系数最高，但 K=4 能够更清晰地区分资源依赖、工业制造、中等转型和经济发达效率型省份，政策解释性更强。因此本文采用 K=4 作为最终分类，结果如表3所示。

表 3 K-means 聚类分类结果

类型	省份	结果分析
资源依赖高排放型	河北、山西、内蒙古、宁夏、新疆	煤炭占比和排放强度高，减排重点是控煤和资源型产业转型
工业制造高排放型	天津、辽宁、江苏、浙江、福建、山东、湖北、广东、重庆、陕西	排放总量较高，减排重点是重点行业节能改造
中等转型压力型	吉林、黑龙江、安徽、江西、河南、湖南、广西、海南、四川、贵州、云南、甘肃、青海	总量和强度居中，需防止高耗能产业转移
经济发达效率型	北京、上海	排放强度低，适合发挥技术、金融和治理优势

4.2 问题二求解结果及分析

根据 2022 年 30 省横截面数据建立 STIRPAT 模型，并采用 OLS 估计参数。主要系数与检验结果见表4。模型 R^2 为 0.8762，调整 R^2 为 0.8504，RMSE_log 为 0.2672，F 检验 p 值为 3.919×10^{-10} ，说明模型对省际碳排放差异具有较强解释力。

表 4 STIRPAT 模型 OLS 估计结果

变量	系数	标准误	t 值	p 值
$\ln P$	0.6447	0.0807	7.9856	0.0000
$\ln A$	0.4502	0.3860	1.1663	0.2549
煤炭相关排放占比	2.8275	0.5798	4.8770	0.0001
第二产业占比	0.5080	1.1516	0.4412	0.6630
城镇化率	0.8029	1.3461	0.5965	0.5565

从参数估计结果看，人口规模和煤炭相关排放占比对碳排放具有显著正向影响。其中， $\ln P$ 系数为 0.6447，说明人口规模扩大将显著增加碳排放；煤炭相关排放占比系数为 2.8275，且 p 值较小，说明能源结构中煤炭依赖程度是影响省际排放差异的关键因素。人均 GDP、第二产业占比和城镇化率系数为正但显著性较弱，这并不意味着这些因素不重要，而是说明在 2022 年横截面样本中，它们的影响可能通过能源结构、产业结构和区域发展阶段间接体现。

VIF 检验结果显示， $\ln P$ 、 $\ln A$ 、煤炭相关排放占比、第二产业占比和城镇化率的 VIF 分别为 1.205、6.568、3.601、2.945 和 5.308，均小于 10，说明不存在严重多重共线性。为进一步检验稳健性，本文采用岭回归进行留一交叉验证，得到 $LOOCV_R^2$ 为 0.7851， $LOOCV_RMSE_log$ 为 0.3520。岭回归标准化系数排序显示，煤炭相关排放占比和人口规模仍是最重要变量，说明核心结论稳定。

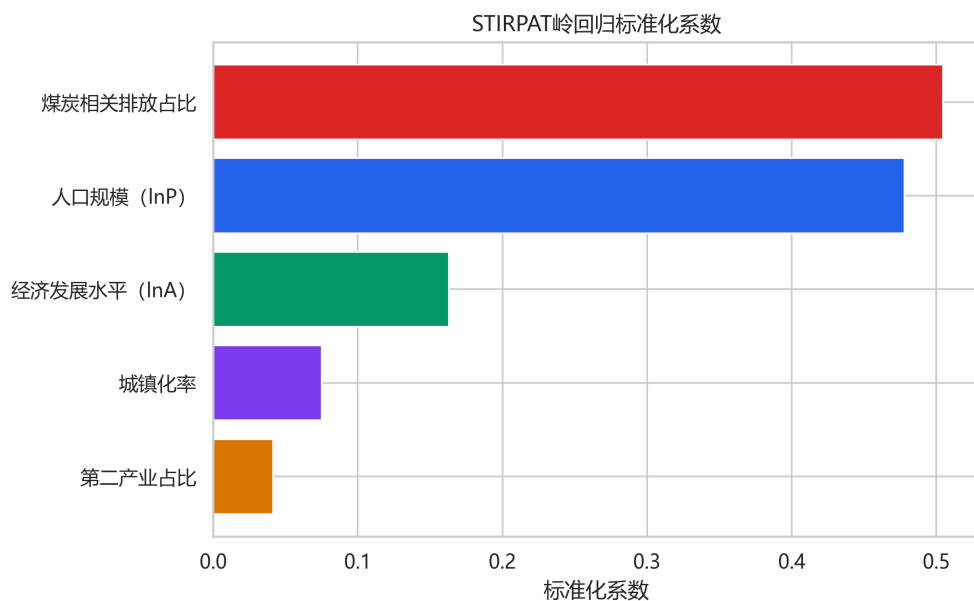


图 3 STIRPAT 岭回归标准化系数

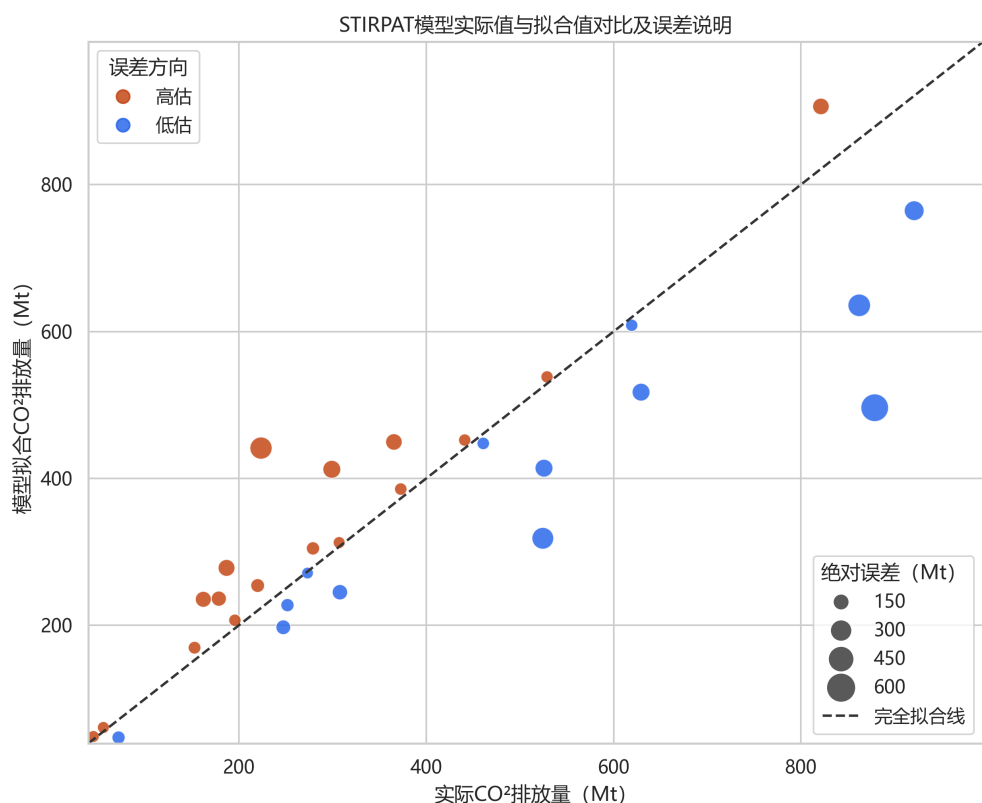


图 4 OLS 拟合值与真实值对比

4.3 问题三求解结果及分析

全国年度碳排放总量仅使用附件 1 中 ‘Sector=Total’ 统计，避免将总量与部门排放重复相加。2019–2024 年全国碳排放整体处于高位波动状态，2023 年达到 11952.40 Mt，2024 年为 11901.44 Mt。2025 年数据只有 273 天记录，不能作为完整年度值进行同比比较。因此，本文以 2024 年全国排放量 11901.44 Mt 作为情景预测基准。

表 5 全国年度 CO₂ 排放核验

年份	CO ₂ 总量/Mt	记录天数	是否完整年份
2019	11148.50	365	是
2020	11193.00	366	是
2021	11619.03	365	是
2022	11627.06	365	是
2023	11952.40	365	是
2024	11901.44	366	是
2025	8615.64	273	否

三种情景的参数设置见表6。表中“起始 → 末期”表示 2025 年至 2045 年按线性插值逐年平滑变化。

表 6 三情景参数设置

情景	人口增长率	人均 GDP 增长率	煤炭占比年变化	第二产业占比年变化	城镇化率年变化
基准情景	-0.10%→-0.20%	4.50%→2.60%	-0.40→-1.00 百分点	-0.05→-0.20 百分点	0.35→0.15 百分点
低碳情景	-0.10%→-0.20%	4.30%→2.40%	-0.60→-1.40 百分点	-0.10→-0.30 百分点	0.30→0.10 百分点
强化低碳情景	-0.10%→-0.25%	4.10%→2.20%	-0.80→-1.80 百分点	-0.20→-0.50 百分点	0.25→0.05 百分点

预测结果见表7和图5。基准情景下，碳排放在 2032 年达到约 12435 Mt 峰值，随后进入缓慢下降阶段，2045 年较峰值下降 11.4%。低碳情景下，排放约在 2026 年达到峰值 11959 Mt，2030 年为 11829 Mt，2045 年较峰值下降 25.6%。强化低碳情景下降幅最大，2045 年降至 7114 Mt，较峰值下降 40.2%。

表 7 三情景达峰与减排潜力

情景	达峰年份	峰值/Mt	2030 年/Mt	2035 年/Mt	2045 年/Mt	2045 较峰值下降
低碳情景	2026	11959	11829	11229	8903	25.6%
基准情景	2032	12435	12404	12351	11014	11.4%
强化低碳情景	2024	11901	11258	10165	7114	40.2%

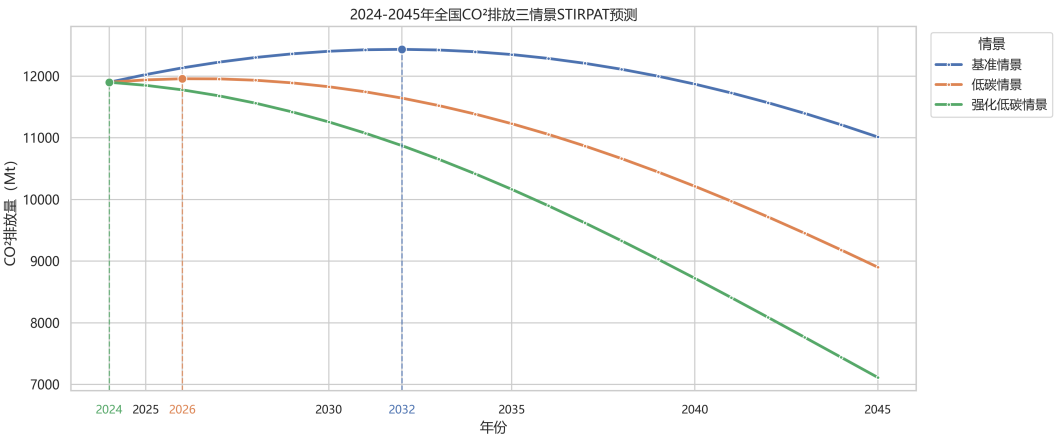


图 5 2024-2045 年三情景碳排放趋势

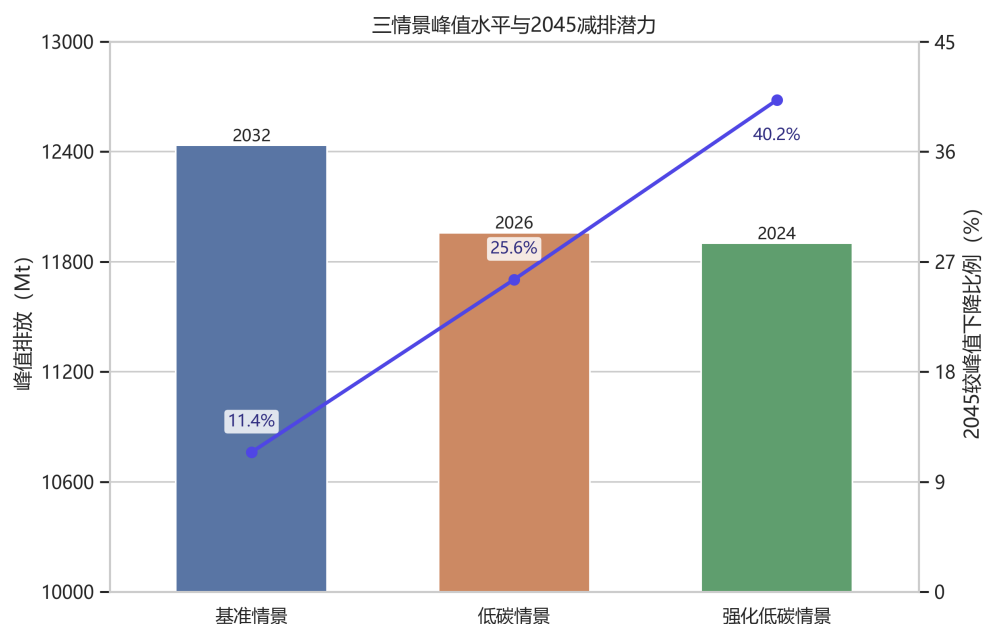


图 6 三情景峰值与减排潜力对比

结果表明，若仅延续基准转型速度，我国可能在 2030 年前后稍晚达峰，峰值水平较高、下降速度较慢；若加快煤炭替代和产业结构优化，则可使达峰时间提前并扩大中长期减排潜力。三种情景下碳排放强度均持续下降，说明经济增长与碳排放逐步脱钩，但不同政策强度下脱钩速度存在明显差异。

灵敏度分析表明，2045 年排放量对煤炭占比下降速度和人均 GDP 增长率最敏感。当煤炭占比下降速度减弱 20% 时，2045 年排放量增加约 954.7 Mt，达峰年份推迟至 2035 年；当煤炭占比下降速度增强 20% 时，2045 年排放量减少约 878.6 Mt，达峰年份提前至 2030 年。人均 GDP 增长率提高 20% 会使 2045 年排放增加约 734.3 Mt，降低 20% 则使 2045 年排放减少约 693.1 Mt。STIRPAT 系数扰动结果也显示，煤炭占比系数和人均 GDP 系数对预测结论影响最大。总体来看，主要结论在合理扰动范围内未发生根本改变，说明模型具有一定稳健性。

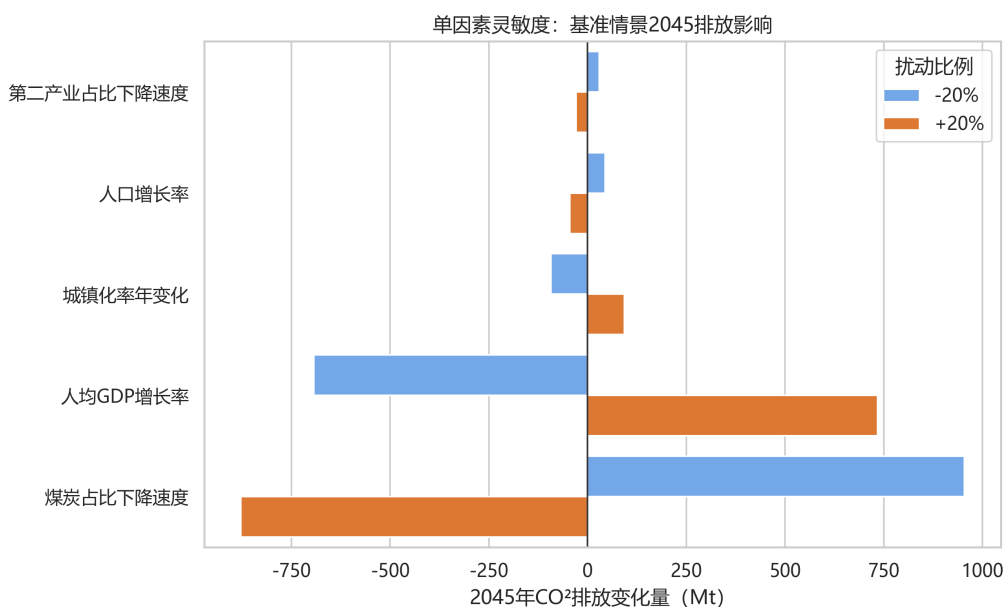


图 7 单因素灵敏度对 2045 年排放的影响

4.4 问题四政策建议

根据前三问结果,本文提出如下政策建议。第一,对资源依赖高排放型省份,应控制煤炭消费总量,推进煤电低碳改造和新能源基地建设,推动资源型产业转型。第二,对工业制造高排放型省份,应聚焦钢铁、建材、电力、石化和交通等重点行业,实施节能改造、电气化替代和绿色供应链管理。第三,对中等转型压力型省份,应在承接产业转移时设置能耗和碳排放准入门槛,防止高耗能产业简单转移。第四,对经济发达效率型地区,应发挥绿色金融、碳交易、数字化碳管理和技术创新优势。第五,从全国层面看,应持续降低煤炭占比,扩大非化石能源供给,强化技术创新和碳市场机制。

5 模型的优缺点及改进方向

5.1 模型优点

第一,模型体系完整。本文从空间差异、综合评价、分类分级、驱动因素识别、趋势预测和政策建议六个层面展开,能够完整回应题目四个问题。第二,解释性较强。熵权 TOPSIS、K-means 聚类 and STIRPAT 模型均具有较清晰的经济含义,便于解释不同省份的低碳发展水平和排放驱动机制。第三,结果展示直观。空间差异指标、TOPSIS 排名、聚类类型、回归系数和情景预测结果都通过表格和图形清晰呈现。第四,情景预测逻辑稳定。STIRPAT 递推模型将问题二估计得到的驱动因素系数直接用于问题三预测,使驱动因素识别与中长期趋势研判形成衔接。

5.2 模型不足

第一,省级建模主要基于 2022 年横截面数据,无法充分刻画各省碳排放随时间变化的动态机制。第二,STIRPAT 模型采用线性对数形式,难以反映复杂非线性关系和变量之间的交互效应。第三,情景预测依赖参数设定,不同参数组合会影响达峰年份和减排潜力判断。第四,当前模型主要分析省际差异,没有进一步考虑相邻地区之间的空间溢出效应。

5.3 改进方向

若能补充多年省级面板数据,可进一步建立双向固定效应 STIRPAT 模型,控制省份个体差异和年份冲击。还可引入空间计量模型,分析碳排放在相邻省份之间的空间相关性和溢出效应。未来也可采用 LMDI 分解进一步量化经济规模、产业结构、能源结构和能源强度对排放变化的贡献,并与 STIRPAT 结果相互验证。在情景预测中,可对煤炭占比下降速度、第二产业占比调整速度、人均 GDP 增长路径等关键参数进行更系统的敏感性分析,提高预测结论的稳健性。

6 参考文献

- [1] Deng H, Yeh C H, Willis R J, Inter-company comparison using modified TOPSIS with objective weights, Computers & Operations Research, 27(10): 963-973, 2000.
- [2] MacQueen J, Some methods for classification and analysis of multivariate observations, Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1: 281-297, 1967.
- [3] York R, Rosa E A, Dietz T, STIRPAT, IPAT and ImPACT: analytic tools for unpacking the driving forces of environmental impacts, Ecological Economics, 46(3): 351-365, 2003.

A 附录：必要计算机程序

本文计算结果由 Python 程序统一生成。附录列出核心程序，包括总入口脚本、公共函数和问题一至问题四的求解脚本。

1.1 总入口脚本 run_all.py

```
1 from __future__ import annotations
2
3 """
4 总入口脚本
5
6 作用：
7 - 依次调用问题一、问题二、问题三、问题四的独立脚本。
8 - 生成分类目录下的结果表、图片、政策建议和汇总报告。
9 - 执行必要的输出完整性检查。
10
11 运行方式：
12 .\\.\venv\Scripts\python.exe .\3. 代码\run_all.py
13 """
14
15 import sys
16 from pathlib import Path
17
18 CODE_ROOT = Path(__file__).resolve().parent
19 ROOT = CODE_ROOT.parent
20 sys.pycache_prefix = str(ROOT / ".pycache")
21 sys.path.append(str(CODE_ROOT))
22
23 import numpy as np
24 import pandas as pd
25
26 from common import (
27     FIG_DIR,
28     PROBLEM_FIG_DIRS,
29     PROBLEM_RESULT_DIRS,
30     RESULT_DIR,
31     ROOT,
32     dataframe_to_markdown,
33     ensure_dirs,
34     percent,
35     read_data,
36     scenario_driver_assumptions,
37     setup_style,
38     write_gitignore,
39 )
40 from 问题一.solve_problem1 import make_problem_one_figures, run_problem_one
41 from 问题二.solve_problem2 import make_problem_two_figures, run_problem_two
42 from 问题三.solve_problem3 import make_problem_three_figures, run_problem_three
43 from 问题四.solve_problem4 import write_policy_report
44
45
46 def write_summary_report(problem_one: dict[str, pd.DataFrame], problem_two: dict[str, pd.DataFrame], problem_three: dict[str,
47 ↪ pd.DataFrame]) -> None:
48     spatial = problem_one["spatial_stats"].set_index(" 指标")
49     topsis = problem_one["topsis"].copy()
50     weights = problem_one["weights"].sort_values(" 权重", ascending=False)
51     silhouette = problem_one["silhouette"].copy()
52     cluster = problem_one["cluster"].copy()
53     cluster_profile = problem_one["cluster_profile"].copy()
54     ols = problem_two["ols_coef"].copy()
55     fit = problem_two["fit"].set_index(" 指标")[" 数值"]
56     vif = problem_two["vif"].copy()
57     ridge = problem_two["ridge_metrics"].set_index(" 指标")[" 数值"]
58     ridge_coef = problem_two["ridge_coef"].sort_values(" 绝对值排序")
59     annual = problem_three["annual"].copy()
60     sector_2024 = problem_three["sector_2024"].copy()
61     peak = problem_three["peak"].copy()
62     stirpat_coef = problem_three["stirpat_coef"].copy()
63     driver_base = problem_three["driver_base"].copy()
64     single_sensitivity = problem_three["single_factor_sensitivity"].copy()
65     coefficient_sensitivity = problem_three["coefficient_sensitivity"].copy()
66
67     top_total = topsis.sort_values("CO2总量_Mt", ascending=False).head(8)[["省份", "CO2总量_Mt", "人均CO2_吨每人",
68 ↪ "碳排放强度_吨每万元GDP"]]
69     top_pc = topsis.sort_values("人均CO2_吨每人", ascending=False).head(5)[["省份", "人均CO2_吨每人", "碳排放强度_吨每万元GDP"]]
70     topsis_top = topsis.sort_values("TOPSIS 低碳得分", ascending=False).head(8)[["省份", "TOPSIS 低碳得分", " 低碳等级"]]
71     topsis_bottom = topsis.sort_values("TOPSIS 低碳得分").head(8)[["省份", "TOPSIS 低碳得分", " 低碳等级"]]
72     cluster_table = (
73         cluster.groupby(" 类型")[" 省份"]
74         .apply(lambda s: " ".join(s.tolist()))
75         .reset_index()
76         .merge(cluster_profile, on=" 类型", how="left")
77     )
78     ols_focus = ols[ols[" 变量"].isin(["lnP", "lnA", " 煤炭相关排放占比", " 第二产业占比", " 城镇化率"])]
79     annual_visible = annual[["Year", "CO2 (Mt)", " 记录天数", " 是否完整年份"]]
80     peak_visible = peak.copy()
81     peak_visible["2045 较峰值下降比例"] = peak_visible["2045 较峰值下降比例"].map(percent)
82     single_focus = single_sensitivity[
```

```

81     single_sensitivity[" 情景"].eq(" 基准情景") & single_sensitivity[" 扰动比例"].isin([-0.20, 0.20])
82 ].copy()
83 single_focus[" 扰动比例"] = single_focus[" 扰动比例"].map(lambda value: f"{value:+.0%}")
84 single_focus[" 影响强度"] = single_focus["2045 变化 _Mt"].abs()
85 single_focus = single_focus.sort_values("影响强度", ascending=False)[["扰动变量", "扰动比例", "达峰年份", "峰值变化_Mt",
    ↪ "2045变化_Mt"]].head(8)
86 coefficient_focus = coefficient_sensitivity[
87     coefficient_sensitivity[" 情景"].eq(" 基准情景") & coefficient_sensitivity[" 扰动比例"].isin([-0.10, 0.10])
88 ].copy()
89 coefficient_focus[" 扰动比例"] = coefficient_focus[" 扰动比例"].map(lambda value: f"{value:+.0%}")
90 coefficient_focus[" 影响强度"] = coefficient_focus["2045 变化 _Mt"].abs()
91 coefficient_focus = coefficient_focus.sort_values("影响强度", ascending=False)[["系数说明", "扰动比例", "达峰年份",
    ↪ "峰值变化_Mt", "2045变化_Mt"]].head(8)
92 assumptions = scenario_driver_assumptions()
93 scenario_param_summary = pd.DataFrame(
94     [
95         {
96             " 情景": scenario,
97             " 人口增长率": f"{values['人口增长率 _ 起始']:.2%}->{values['人口增长率 _ 末期']:.2%}",
98             " 人均 GDP 增长率": f"{values['人均 GDP 增长率 _ 起始']:.2%}->{values['人均 GDP 增长率 _ 末期']:.2%}",
99             "煤炭占比年变化": f"{values['煤炭占比年变化_起始'] * 100:.2f}->{values['煤炭占比年变化_末期'] * 100:.2f}个百分点",
100            "第二产业占比年变化": f"{values['第二产业占比年变化_起始'] * 100:.2f}->{values['第二产业占比年变化_末期'] *
    ↪ 100:.2f}个百分点",
101            "城镇化率年变化": f"{values['城镇化率年变化_起始'] * 100:.2f}->{values['城镇化率年变化_末期'] * 100:.2f}个百分点",
102        }
103        for scenario, values in assumptions.items()
104    ]
105 )
106
107 total_stats = spatial.loc["CO2 总量 _Mt"]
108 pc_stats = spatial.loc[" 人均 CO2 _ 吨每人"]
109 intensity_stats = spatial.loc[" 碳排放强度 _ 吨每万元 GDP"]
110 coal_stats = spatial.loc[" 煤炭相关排放占比"]
111 best_k = int(silhouette.sort_values(" 轮廓系数", ascending=False).iloc[0]["K"])
112 best_k_score = float(silhouette.sort_values(" 轮廓系数", ascending=False).iloc[0][" 轮廓系数"])
113 base_peak = peak.set_index(" 情景").loc[" 基准情景"]
114 low_peak = peak.set_index(" 情景").loc[" 低碳情景"]
115 strong_peak = peak.set_index(" 情景").loc[" 强化低碳情景"]
116
117 summary = f"### B 题结果汇总
118
119 ## 1. 数据口径与总说明
120
121 本项目使用`数据/2022省级综合表.xlsx`完成30省横截面分析, 使用`数据/附件1: 2019-2025
    ↪ 年全国碳排放数据.csv`中`Sector=Total`完成全国趋势分析。附件1中2025年只有截至2025-09-30的数据, 不能直接作为全年值与2019至2024年比较;
    ↪ 因此三情景预测以2024年全国排放量{annual.loc[annual['Year'].eq(2024), 'CO2 (Mt)'].iloc[0]:.2f} Mt作为基准。
122
123 ## 2. 问题一: 碳排放空间差异分析与省份分类分级
124
125 > 题目要求: " 选取 ** 碳排放总量、人均碳排放、碳排放强度等 ** 核心指标, 分析各省碳排放空间分布特征与差异显著性。"
126
127 解答: 结合省级CO2总量空间分布热力图可知, 2022年我国30省碳排放呈现明显的地理集聚特征, 而不是均匀分布。第一,
    ↪ 高排放省份集中在华北-环渤海及内蒙古能源基地一带, 山东、内蒙古、河北、山西和辽宁形成北方高排放集聚区, 其中山东、河北、辽宁与钢铁、石化、
    ↪ 装备制造等重工业基础有关, 内蒙古、山西则与煤炭资源开发和能源外送密切相关。第二, 东部沿海制造业省份排放总量同样较高, 江苏、广东、浙江、
    ↪ 福建等省沿海成带分布, 说明经济规模、工业制造和外向型产业集聚显著抬高了排放总量。第三, 中西部和西南多数省份排放总量处于中低水平,
    ↪ 但新疆因能源资源开发排放较高, 表现为西北地区中的局部高值; 青海、海南、北京等省市排放总量较低, 与人口规模、产业结构和生态功能定位有关。
    ↪ 整体来看, 空间格局可概括为"北方资源能源高排放带、东部沿海制造业高排放带、中西南相对低排放区、局部资源型高值点"。
128
129 差异显著性方面, 2022年30省CO2总量均值为{total_stats['均值']:.2f} Mt, 但最大值山东为920.91 Mt, 最小值海南为44.69 Mt, 二者相差约20.61倍;
    ↪ 排放总量前8位省份合计占30省总排放的50.72%, 说明排放明显集中在少数重点省份。统计指标进一步验证了这种空间不均衡:
    ↪ CO2总量变异系数为{total_stats['变异系数']:.4f}、基尼系数为{total_stats['基尼系数']:.4f}、泰尔指数为{total_stats['泰尔指数']:.4f},
    ↪ 表明省际排放总量差异显著。人均CO2变异系数为{pc_stats['变异系数']:.4f}, 碳排放强度变异系数为{intensity_stats['变异系数']:.4f}、
    ↪ 基尼系数为{intensity_stats['基尼系数']:.4f}, 说明资源型地区和重工业地区不仅排放总量高, 单位人口和单位经济产出的排放压力也更突出。
    ↪ 煤炭相关排放占比均值为{coal_stats['均值']:.4f}, 说明煤炭依赖具有普遍性; 但其基尼系数仅为{coal_stats['基尼系数']:.4f},
    ↪ 低于碳排放强度基尼系数, 说明各省能源结构差异虽存在, 但真正拉开区域低碳发展差距的是能源结构、
    ↪ 产业结构和经济效率共同作用下形成的排放强度差异。
130
131 高排放总量省份主要为:
132
133 {dataframe_to_markdown(top_total, ".3f")}
134
135 人均排放压力最高的省份主要为:
136
137 {dataframe_to_markdown(top_pc, ".3f")}
138
139 > 题目要求: "综合排放规模、排放效率、经济关联度构建多维度评价体系, 建立分类分级模型, 对全国各省碳排放水平进行科学分类与等级划分,
    ↪ 并说明分类依据与政策适配性。"
140
141 解答: 本文以CO2总量、人均CO2、碳排放强度、煤炭相关排放占比、第二产业占比、人均GDP、城镇化率7个指标构建评价体系,
    ↪ 采用熵权TOPSIS计算低碳发展综合得分。权重最高的三个指标是{weights.iloc[0]['指标']} ( {weights.iloc[0]['权重']:.4f} )、
    ↪ {weights.iloc[1]['指标']} ( {weights.iloc[1]['权重']:.4f} )、{weights.iloc[2]['指标']} ( {weights.iloc[2]['权重']:.4f} ),
    ↪ 说明经济发展质量、煤炭依赖程度和城镇化治理能力对综合评价影响较大。
142
143 TOPSIS 排名靠前省份为:
144
145 {dataframe_to_markdown(topsis_top, ".3f")}
146
147 TOPSIS 排名靠后省份为:
148
149 {dataframe_to_markdown(topsis_bottom, ".3f")}
150

```

聚类模型方面，K=2至6的轮廓系数显示K={best_k}时统计分离度最高（轮廓系数{best_k_score:.4f}）。但为增强政策解释性，主模型采用K=4，
 ↳ 将省份划分为资源依赖高排放型、工业制造高排放型、中等转型压力型和经济发达效率型。分类结果如下：

```
{dataframe_to_markdown(cluster_table, ".3f")}
```

政策适配性：资源依赖高排放型应重点控煤和推动资源型产业转型；工业制造高排放型应聚焦钢铁、建材、电力等行业节能改造；
 ↳ 中等转型压力型应避免高耗能产业简单转移；经济发达效率型应发挥技术、金融和治理优势。

3. 问题二：碳排放影响因素识别与预测模型构建

> 题目要求："选取 ** 能源消费结构、产业结构、人均 GDP、城镇化率、技术水平等 ** 关键指标，分析影响碳排放的核心驱动因素。"

解答：基于现有横截面数据，构建STIRPAT模型：因变量为ln(CO2总量)，解释变量为ln(人口)、ln(人均GDP)、煤炭相关排放占比、第二产业占比、城镇化率。OLS估计结果显示，ln(人口)系数为{ols_focus.loc[ols_focus['变量'].eq('lnP')].eq('lnP')}, '系数'.iloc[0]:.4f},
 ↳ t值为{ols_focus.loc[ols_focus['变量'].eq('lnP')].eq('lnP')}.iloc[0]:.2f};
 ↳ 煤炭相关排放占比系数为{ols_focus.loc[ols_focus['变量'].eq('煤炭相关排放占比')].eq('煤炭相关排放占比')}, '系数'.iloc[0]:.4f},
 ↳ t值为{ols_focus.loc[ols_focus['变量'].eq('煤炭相关排放占比')].eq('煤炭相关排放占比')}.iloc[0]:.2f}。二者是当前样本中最显著、最稳定的正向驱动因素。
 ↳ 人均GDP、第二产业占比和城镇化率系数为正，但显著性较弱，说明其影响可能通过产业结构、能源结构和区域发展阶段间接体现。

OLS 主要系数如下：

```
{dataframe_to_markdown(ols_focus, ".4f")}
```

> 题目要求："构建碳排放定量预测模型，完成模型参数估计、显著性检验与优化，确保模型具有良好拟合效果与预测精度，并说明模型的适用范围与局限性。"
 ↳ "

解答：OLS模型R2为{fit['R2']:.4f}，调整R2为{fit['调整R2']:.4f}，RMSE_log为{fit['RMSE_log']:.4f}，F检验p值为{fit['F检验p值']:.4g},
 ↳ 说明模型对2022年省际排放差异具有较强解释力。VIF检验结果如下，所有变量VIF均低于10，不存在严重多重共线性：

```
{dataframe_to_markdown(vif, ".3f")}
```

为提升稳健性，进一步采用岭回归和留一交叉验证。岭回归最优alpha为{ridge['最优alpha']:.4f}，LOOCV_R2为{ridge['LOOCV_R2']:.4f},
 ↳ LOOCV_RMSE_log为{ridge['LOOCV_RMSE_log']:.4f}。标准化系数排序显示，
 ↳ {ridge_coef.iloc[0]['变量']}和{ridge_coef.iloc[1]['变量']}仍是最重要变量，说明"人口规模+煤炭依赖"的结论稳定。
 ↳ 模型适用于解释2022年省际横截面差异；局限是当前缺少多年省级面板数据，难以直接识别长期动态因果关系。

4. 问题三：多情景碳排放趋势预测与碳达峰研判

> 题目要求："基于最优预测模型，设定 ** 基准、低碳、强化低碳 ** 三种发展情景，预测 2026-2045 年全国碳排放总量与强度变化趋势。"

解答：由于全国年度样本较短，本文不直接建立全国多变量时间序列回归，而是采用问题二估计得到的STIRPAT系数进行相对递推。具体形式为：
 ↳ 以2024年全国CO2排放为锚点，按'DeltaInC = beta1DeltaInP + beta2DeltaInA + beta3DeltaCoal + beta4DeltaIS + beta5DeltaU'计算下一年排放变化，其中P为人口规模，A为人均GDP，Coal为煤炭相关排放占比，IS为第二产业占比，U为城镇化率。
 ↳ 附件1年度排放核验如下，其中2025年为不完整年份：

```
{dataframe_to_markdown(annual_visible, ".2f")}
```

用于问题三递推的 STIRPAT 系数如下：

```
{dataframe_to_markdown(stirpat_coeff, ".4f")}
```

全国驱动变量基准值如下：

```
{dataframe_to_markdown(driver_base, ".4f")}
```

三种情景参数设置如下。人口增长率和人均GDP增长率为年度相对增长率；煤炭占比、第二产业占比和城镇化率为年度占比变化量，单位为百分点。
 ↳ 表中"起始->末期"表示2025年至2045年按线性插值逐年平滑变化：

```
{dataframe_to_markdown(scenario_param_summary, ".4f")}
```

2024 年分部门排放显示，工业、电力和地面交通是主要排放来源：

```
{dataframe_to_markdown(sector_2024.head(6), ".2f")}
```

> 题目要求："研判碳达峰时间节点、峰值水平及减排潜力，为'双碳'目标实施提供量化支撑。"

解答：三情景预测结果如下：

```
{dataframe_to_markdown(peak_visible, ".3f")}
```

基准情景下，全国排放将在{int(base_peak['达峰年份'])}年达到峰值{base_peak['峰值_Mt']:.0f} Mt，之后缓慢下降，
 ↳ 2045年为{base_peak['2045年_Mt']:.0f} Mt，较峰值下降{base_peak['2045较峰值下降比例']:.1%}。低碳情景下，
 ↳ 全国排放将在{int(low_peak['达峰年份'])}年附近达峰，2030年为{low_peak['2030年_Mt']:.0f} Mt，
 ↳ 2045年较峰值下降{low_peak['2045较峰值下降比例']:.1%}。强化低碳情景下降幅最大，2045年降至{strong_peak['2045年_Mt']:.0f} Mt，
 ↳ 较峰值下降{strong_peak['2045较峰值下降比例']:.1%}。从碳排放强度看，2045年基准、
 ↳ 低碳和强化低碳情景较2024年分别下降{base_peak['2045较2024强度下降比例']:.1%}、
 ↳ {low_peak['2045较2024强度下降比例']:.1%}和{strong_peak['2045较2024强度下降比例']:.1%}。因此，煤炭占比下降、
 ↳ 产业结构优化和人口经济变量的共同变化，是决定达峰时点和中长期减排潜力的关键。

灵敏度分析方面，方案一对人口增长率、人均GDP增长率、煤炭占比下降速度、第二产业占比下降速度和城镇化率年变化分别进行±10%、±20%单因素扰动；
 ↳ 方案四对STIRPAT核心系数进行±10%扰动。这里"下降速度+20%"表示负向占比变化的绝对幅度增大，即占比下降更快。基准情景下，
 ↳ 单因素扰动对2045年排放影响最大的变量如下：

```
{dataframe_to_markdown(single_focus, ".3f")}
```

STIRPAT 系数扰动结果如下：

```
{dataframe_to_markdown(coefficient_focus, ".3f")}
```

结果显示，2045年排放对煤炭占比变化和人均GDP增长最敏感；在模型系数层面，对煤炭占比系数和人均GDP系数最敏感。
 ↳ 主要情景结论没有因小幅扰动而发生根本改变，说明模型具有一定稳健性。


```

216
217 ## 5. 问题四：政策建议
218
219 > 题目要求：“结合问题一至三的研究结论，针对‘双碳’目标实现路径，从优化能源结构、推动产业升级、实施区域差异化减排、强化技术创新、
↳ 完善政策机制等方面，提出科学可行、针对性强、可操作性高的政策建议。”

220
221 解答：政策建议已单独写入`1. 结果/问题四/问题四_政策建议报告.md`。核心建议是按省份类型实施差异化减排：
↳ 资源依赖高排放型控煤并推动资源型产业转型，工业制造高排放型聚焦重点行业节能降碳，中等转型压力型强化产业准入和效率提升，
↳ 经济发达效率型发挥绿色金融、技术创新和碳管理优势。全国层面应持续降低煤炭占比、扩大非化石能源供给、强化碳市场和技术创新，
↳ 因为STIRPAT情景递推结果表明，煤炭占比下降和产业结构优化越快，峰值越低、达峰后下降越明显。

222
223 ## 6. 文件索引
224
225 - 问题一结果：`1. 结果/问题一/01_ 空间差异指标.csv`等 6 个 CSV 文件
226 - 问题二结果：`1. 结果/问题二/01_OLS 系数.csv`等 6 个 CSV 文件
227 - 问题三结果：`1. 结果/问题三/01_ 全国年度排放.csv`等 9 个 CSV 文件
228 - 问题四报告：`1. 结果/问题四/问题四_ 政策建议报告.md`
229 - 图片目录：`2. 图片/问题一`、`2. 图片/问题二`、`2. 图片/问题三`
230 - 代码目录：`3. 代码/问题一`、`3. 代码/问题二`、`3. 代码/问题三`、`3. 代码/问题四`
231
232 out_dir = PROBLEM_RESULT_DIRS[" 汇总"]
233 out_dir.mkdir(parents=True, exist_ok=True)
234 (out_dir / "B 题结果汇总.md").write_text(summary, encoding="utf-8")
235
236
237 def write_readme() -> None:
238     readme = ""
239     # B 题：我国碳排放时空特征分析与趋势预测
240
241     本项目为第十届校内数学建模竞赛 B 题的数据处理、模型求解、图表生成和论文素材整理工程。
242
243 ## 项目结构
244
245 ```text
246 数据处理/
247 | - 数据/ # 题目附件与整理后的数据
248 | - 题目/ # B 题目文件
249 | - 1. 结果/
250 | | - 问题一/ # 空间差异、TOPSIS、聚类结果
251 | | - 问题二/ # STIRPAT 回归、检验、岭回归结果
252 | | - 问题三/ # 三情景预测和达峰结果
253 | | - 问题四/ # 政策建议报告
254 | - 汇总/ # B 题结果汇总
255 | - 2. 图片/
256 | | - 问题一/ # 空间分布、TOPSIS、聚类图
257 | | - 问题二/ # STIRPAT 系数和 OLS 拟合图
258 | | - 问题三/ # 情景预测和达峰图
259 | - 3. 代码/
260 | | - assets/china_provinces.geojson
261 | | - 问题一/solve_problem1.py
262 | | - 问题二/solve_problem2.py
263 | | - 问题三/solve_problem3.py
264 | | - 问题四/solve_problem4.py
265 | | - common.py
266 | | - run_all.py
267 | - requirements.txt
268 | - README.md
269 ```
270
271 ## 环境安装
272
273 ```powershell
274 python -m venv .venv
275 .\\.venv\\Scripts\\python.exe -m pip install -r .\\3. 代码\\requirements.txt
276 ```
277
278 ## 一键运行
279
280 ```powershell
281 .\\.venv\\Scripts\\python.exe .\\3. 代码\\run_all.py
282 ```
283
284 也可以单独运行某个问题：
285
286 ```powershell
287 .\\.venv\\Scripts\\python.exe .\\3. 代码\\问题一\\solve_problem1.py
288 .\\.venv\\Scripts\\python.exe .\\3. 代码\\问题二\\solve_problem2.py
289 .\\.venv\\Scripts\\python.exe .\\3. 代码\\问题三\\solve_problem3.py
290 .\\.venv\\Scripts\\python.exe .\\3. 代码\\问题四\\solve_problem4.py
291 ```
292
293 运行后会自动生成或覆盖同名结果文件，不会清空目录或删除原始数据。
294
295 ## 数据说明
296
297 - `数据/附件1：2019-2025 年全国碳排放数据.csv`：全国碳排放时间序列，字段为`Area, CO2 (Mt), Sector, Date`。全国总量分析只使用`Sector ==
↳ "Total"`。
298 - `数据/附件 2：2022 年全国 30 个省份碳排放清单.xlsx`：30 省分能源、分部门碳排放清单。
299 - `数据/2022 省级综合表.xlsx`：已合并人口、GDP、第二产业、城镇化率、排放强度和能源结构指标，是问题一和问题二主表。
300 - `数据/全国数据.xlsx`：2019 至 2024 年全国 GDP、人口、能源消费总量和煤炭占比，用于辅助情景设定。
301 - `3. 代码/assets/china_provinces.geojson`：省级行政边界文件，用于绘制问题一省级 CO2 总量空间分布热力图。

```

```

302 注意：附件 1 中 2025 年数据截至 2025-09-30，不能直接作为全年值与 2019 至 2024 年比较。三情景预测以 2024 年为基准年。
303
304 ## 模型路线
305
306 1. 问题一：变异系数、基尼系数、泰尔指数、熵权 TOPSIS、K-means 聚类，并结合省级边界绘制 CO2 总量空间分布热力图。
307 2. 问题二：STIRPAT 横截面 OLS、VIF 共线性检验、岭回归留一交叉验证。
308 3. 问题三：STIRPAT 系数驱动的三情景递推预测、达峰年份和减排潜力判断。
309 4. 问题四：根据分类、驱动因素和情景预测生成差异化政策建议。
310
311 ## 输出说明
312
313 - `1. 结果/汇总/B 题结果汇总.md`：按题目原问题逐条引用并给出详细解答。
314 - `1. 结果/问题一/`：空间差异、TOPSIS 和聚类 CSV 结果。
315 - `1. 结果/问题二/`：OLS、VIF、岭回归和拟合残差 CSV 结果。
316 - `1. 结果/问题三/`：年度排放、情景参数、STIRPAT 递推系数、预测序列、达峰结果和灵敏度分析。
317 - `1. 结果/问题四/`：约 500 字政策建议报告。
318 - `2. 图片/`：按问题分文件夹保存 PNG 图片。
319
320 ## 可复现性说明
321
322 所有结果由`3.代码/run_all.py`从`数据/`目录重新计算得到。若需要调整情景参数，
323 ↩ 可修改`3.代码/common.py`中的`scenario_driver_assumptions()`后重新运行。
324
325 """
326 (ROOT / "README.md").write_text(readme, encoding="utf-8")
327
328 def validate_outputs(problem_one: dict[str, pd.DataFrame], problem_two: dict[str, pd.DataFrame], problem_three: dict[str,
329 ↩ pd.DataFrame]) -> None:
330     topsis = problem_one["topsis"]
331     if topsis["TOPSIS 低碳得分"].isna().any() or not topsis["TOPSIS 低碳得分"].between(0, 1).all():
332         raise AssertionError("TOPSIS 得分应非空且位于 0 到 1 之间。")
333     if problem_one["cluster"]["省份"].nunique() != 30:
334         raise AssertionError("K-means 聚类结果应覆盖 30 个省份。")
335     if np.isinf(problem_two["vif"]["VIF"]).any():
336         raise AssertionError("VIF 结果存在无限值。")
337     forecast = problem_three["forecast"]
338     for scenario, group in forecast.groupby("情景"):
339         if not set(range(2026, 2046)).issubset(set(group["年份"])):
340             raise AssertionError(f"{scenario} 未覆盖 2026-2045 完整预测区间。")
341     expected_csvs = {
342         "问题一": ["01_空间差异指标", "02_熵权指标权重", "03_TOPSIS得分排名", "04_聚类K值检验", "05_K4聚类结果", "06_类型均值画像"],
343         "问题二": ["01_OLS系数", "02_拟合优度", "03_VIF共线性检验", "04_拟合值残差", "05_岭回归交叉验证", "06_岭回归标准化系数"],
344         "问题三": [
345             "01_ 全国年度排放",
346             "02_2024 部门排放",
347             "03_ 情景参数",
348             "04_2024-2045 预测序列",
349             "05_ 达峰与减排潜力",
350             "06_STIRPAT 递推系数",
351             "07_ 驱动变量基准值",
352             "08_ 单因素灵敏度分析",
353             "09_STIRPAT 系数灵敏度分析",
354         ],
355     }
356     for problem, stems in expected_csvs.items():
357         for stem in stems:
358             path = PROBLEM_RESULT_DIRS[problem] / f"{stem}.csv"
359             if not path.exists() or path.stat().st_size == 0:
360                 raise AssertionError(f"CSV 结果未正确生成: {path}")
361     expected_figs = {
362         "问题一": [
363             "01_ 省份 CO2 总量排名图",
364             "02_ 人均 CO2 与碳排放强度散点图",
365             "03_TOPSIS 低碳得分排名图",
366             "04_1_ 资源依赖高排放型 _ 均值画像雷达图",
367             "04_2_ 工业制造高排放型 _ 均值画像雷达图",
368             "04_3_ 中等转型压力型 _ 均值画像雷达图",
369             "04_4_ 经济发达效率型 _ 均值画像雷达图",
370             "05_ 省级 CO2 总量空间分布热力图",
371         ],
372         "问题二": ["05_STIRPAT 标准化系数图", "06_OLS 拟合值与真实值对比图"],
373         "问题三": [
374             "07_ 三情景碳排放趋势图",
375             "08_ 三情景峰值与减排潜力对比图",
376             "09_ 单因素灵敏度 _2045 排放影响图",
377             "10_STIRPAT 系数灵敏度 _2045 排放影响图",
378         ],
379     }
380     for problem, stems in expected_figs.items():
381         for stem in stems:
382             path = PROBLEM_FIG_DIRS[problem] / f"{stem}.png"
383             if not path.exists() or path.stat().st_size == 0:
384                 raise AssertionError(f"图片未正确生成: {path}")
385
386 def main() -> None:
387     ensure_dirs()
388     setup_style()
389     province, national, carbon = read_data()
390     problem_one = run_problem_one(province)
391     problem_two = run_problem_two(province)
392     problem_three = run_problem_three(carbon, national, province, problem_two)

```

```

392     make_problem_one_figures(problem_one)
393     make_problem_two_figures(problem_two)
394     make_problem_three_figures(problem_three)
395     write_policy_report(problem_one, problem_two, problem_three)
396     write_summary_report(problem_one, problem_two, problem_three)
397     write_readme()
398     write_gitignore()
399     validate_outputs(problem_one, problem_two, problem_three)
400     print(" 全部结果已按问题分类生成。")
401     print(f" 结果目录: {RESULT_DIR}")
402     print(f" 图片目录: {FIG_DIR}")
403
404
405 if __name__ == "__main__":
406     main()

```

1.2 公共函数 common.py

```

1  from __future__ import annotations
2
3  import math
4  import warnings
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10 import seaborn as sns
11 from matplotlib import font_manager
12
13
14 warnings.filterwarnings("ignore", category=UserWarning)
15
16 ROOT = Path(__file__).resolve().parents[1]
17 DATA_DIR = ROOT / " 数据"
18 RESULT_DIR = ROOT / "1. 结果"
19 FIG_DIR = ROOT / "2. 图片"
20 CODE_DIR = ROOT / "3. 代码"
21
22 PROBLEM_RESULT_DIRS = {
23     " 问题一": RESULT_DIR / " 问题一",
24     " 问题二": RESULT_DIR / " 问题二",
25     " 问题三": RESULT_DIR / " 问题三",
26     " 问题四": RESULT_DIR / " 问题四",
27     " 汇总": RESULT_DIR / " 汇总",
28 }
29
30 PROBLEM_FIG_DIRS = {
31     " 问题一": FIG_DIR / " 问题一",
32     " 问题二": FIG_DIR / " 问题二",
33     " 问题三": FIG_DIR / " 问题三",
34     " 问题四": FIG_DIR / " 问题四",
35 }
36
37 PROVINCE_FILE = DATA_DIR / "2022 省级综合表.xlsx"
38 NATIONAL_FILE = DATA_DIR / " 全国数据.xlsx"
39 CARBON_FILE = DATA_DIR / " 附件 1: 2019-2025 年全国碳排放数据.csv"
40
41
42 def ensure_dirs() -> None:
43     RESULT_DIR.mkdir(exist_ok=True)
44     FIG_DIR.mkdir(exist_ok=True)
45     for path in [*PROBLEM_RESULT_DIRS.values(), *PROBLEM_FIG_DIRS.values()]:
46         path.mkdir(parents=True, exist_ok=True)
47
48
49 def setup_style() -> None:
50     sns.set_theme(style="whitegrid", context="notebook")
51     candidates = [
52         "Microsoft YaHei",
53         "SimHei",
54         "Noto Sans CJK SC",
55         "Source Han Sans SC",
56         "Arial Unicode MS",
57     ]
58     available = {f.name for f in font_manager.fontManager.ttflist}
59     for font in candidates:
60         if font in available:
61             plt.rcParams["font.sans-serif"] = [font]
62             break
63     plt.rcParams["axes.unicode_minus"] = False
64     plt.rcParams["figure.dpi"] = 150
65     plt.rcParams["savefig.dpi"] = 300
66
67
68 def save_fig(fig: plt.Figure, output_dir: Path, stem: str) -> None:
69     fig.tight_layout()
70     output_dir.mkdir(parents=True, exist_ok=True)
71     fig.savefig(output_dir / f"{stem}.png", bbox_inches="tight")
72     plt.close(fig)

```

```

73
74
75 def read_data() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
76     province = pd.read_excel(PROVINCE_FILE, sheet_name=" 主表")
77     national = pd.read_excel(NATIONAL_FILE)
78     carbon = pd.read_csv(CARBON_FILE, encoding="utf-8-sig")
79     carbon["Date"] = pd.to_datetime(carbon["Date"])
80     carbon["Year"] = carbon["Date"].dt.year
81     return province, national, carbon
82
83
84 def require_columns(df: pd.DataFrame, columns: list[str]) -> None:
85     missing = [col for col in columns if col not in df.columns]
86     if missing:
87         raise ValueError(f" 缺少必要字段: {missing}")
88     if df[columns].isna().any().any():
89         missing_counts = df[columns].isna().sum()
90         raise ValueError(f" 关键字段存在缺失值:\n{missing_counts[missing_counts > 0]}")
91
92
93 def gini(values: pd.Series | np.ndarray) -> float:
94     arr = np.sort(np.asarray(values, dtype=float))
95     n = len(arr)
96     if n == 0 or arr.sum() == 0:
97         return np.nan
98     return float((2 * np.arange(1, n + 1) @ arr) / (n * arr.sum()) - (n + 1) / n)
99
100
101 def theil(values: pd.Series | np.ndarray) -> float:
102     arr = np.asarray(values, dtype=float)
103     mean = arr.mean()
104     if mean == 0:
105         return np.nan
106     ratio = arr / mean
107     ratio = ratio[ratio > 0]
108     return float(np.mean(ratio * np.log(ratio)))
109
110
111 def dataframe_to_markdown(df: pd.DataFrame, floatfmt: str = ".3f") -> str:
112     headers = [str(col) for col in df.columns]
113     rows = []
114     for _, row in df.iterrows():
115         values = []
116         for value in row:
117             if isinstance(value, (float, np.floating)):
118                 values.append(format(float(value), floatfmt))
119             else:
120                 values.append(str(value))
121         rows.append(values)
122     lines = [
123         "| " + " | ".join(headers) + " |",
124         "| " + " | ".join(["---"] * len(headers)) + " |",
125     ]
126     lines.extend("| " + " | ".join(values) + " |" for values in rows)
127     return "\n".join(lines)
128
129
130 def write_csv(df: pd.DataFrame, path: Path) -> None:
131     path.parent.mkdir(parents=True, exist_ok=True)
132     df.to_csv(path, index=False, encoding="utf-8-sig")
133
134
135 def write_gitignore() -> None:
136     gitignore = ROOT / ".gitignore"
137     desired = [
138         ".venv/",
139         ".pycache/",
140         "__pycache__/",
141         "*.pyc",
142         ".pytest_cache/",
143         ".mypy_cache/",
144         ".ipynb_checkpoints/",
145         "-*.xlsx",
146         "-*.docx",
147     ]
148     existing = gitignore.read_text(encoding="utf-8").splitlines() if gitignore.exists() else []
149     merged = existing[:]
150     for line in desired:
151         if line not in merged:
152             merged.append(line)
153     gitignore.write_text("\n".join(merged).rstrip() + "\n", encoding="utf-8")
154
155
156 def percent(value: float) -> str:
157     return f"{value:.1%}"
158
159
160 def scenario_driver_assumptions() -> dict[str, dict[str, float]]:
161     return {
162         " 基准情景": {
163             " 人口增长率 _ 起始": -0.0010,
164             " 人口增长率 _ 末期": -0.0020,

```

```

165         " 人均 GDP 增长率 _ 起始": 0.0450,
166         " 人均 GDP 增长率 _ 末期": 0.0260,
167         " 煤炭占比年变化 _ 起始": -0.0040,
168         " 煤炭占比年变化 _ 末期": -0.0100,
169         " 第二产业占比年变化 _ 起始": -0.0005,
170         " 第二产业占比年变化 _ 末期": -0.0020,
171         " 城镇化率年变化 _ 起始": 0.0035,
172         " 城镇化率年变化 _ 末期": 0.0015,
173     },
174     " 低碳情景": {
175         " 人口增长率 _ 起始": -0.0010,
176         " 人口增长率 _ 末期": -0.0020,
177         " 人均 GDP 增长率 _ 起始": 0.0430,
178         " 人均 GDP 增长率 _ 末期": 0.0240,
179         " 煤炭占比年变化 _ 起始": -0.0060,
180         " 煤炭占比年变化 _ 末期": -0.0140,
181         " 第二产业占比年变化 _ 起始": -0.0010,
182         " 第二产业占比年变化 _ 末期": -0.0030,
183         " 城镇化率年变化 _ 起始": 0.0030,
184         " 城镇化率年变化 _ 末期": 0.0010,
185     },
186     " 强化低碳情景": {
187         " 人口增长率 _ 起始": -0.0010,
188         " 人口增长率 _ 末期": -0.0025,
189         " 人均 GDP 增长率 _ 起始": 0.0410,
190         " 人均 GDP 增长率 _ 末期": 0.0220,
191         " 煤炭占比年变化 _ 起始": -0.0080,
192         " 煤炭占比年变化 _ 末期": -0.0180,
193         " 第二产业占比年变化 _ 起始": -0.0020,
194         " 第二产业占比年变化 _ 末期": -0.0050,
195         " 城镇化率年变化 _ 起始": 0.0025,
196         " 城镇化率年变化 _ 末期": 0.0005,
197     },
198 }

```

1.3 问题一求解脚本 solve_problem1.py

```

1  from __future__ import annotations
2
3  """
4  问题一脚本说明
5
6  算法与模型：
7  - 变异系数、基尼系数、泰尔指数：度量 30 省碳排放空间差异和离散程度。
8  - 熵权 TOPSIS：构建低碳发展综合评价得分，指标包括排放规模、排放效率、能源结构、产业结构和经济发展水平。
9  - K-means 聚类：按标准化后的 7 个指标将省份划分为 4 类，并计算 K=2 至 6 的轮廓系数作为检验。
10
11  使用数据：
12  - 输入：数据/2022 省级综合表.xlsx，工作表" 主表"。
13  - 辅助边界：3. 代码/assets/china_provinces.geojson，用于绘制省级空间分布热力图。
14  - 输出：1. 结果/问题一/ 下多个 CSV 结果文件。
15  - 输出图片：2. 图片/问题一/01 至 05 号图。
16  """
17
18  import sys
19  from pathlib import Path
20
21  CODE_ROOT = Path(__file__).resolve().parents[1]
22  ROOT = CODE_ROOT.parent
23  sys.pycache_prefix = str(ROOT / ".pycache")
24  sys.path.append(str(CODE_ROOT))
25
26  import json
27  import matplotlib.pyplot as plt
28  from matplotlib.collections import PatchCollection
29  import matplotlib.patheffects as path_effects
30  from matplotlib.patches import Patch
31  from matplotlib.patches import Polygon as MplPolygon
32  import numpy as np
33  import pandas as pd
34  import seaborn as sns
35  from sklearn.cluster import KMeans
36  from sklearn.metrics import silhouette_score
37  from sklearn.preprocessing import StandardScaler
38
39  from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, gini, read_data, require_columns, save_fig, setup_style, theil,
40  ↪ write_csv
41
42  PROBLEM = " 问题一"
43  RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
44  FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
45  CHINA_GEOJSON = CODE_ROOT / "assets" / "china_provinces.geojson"
46
47
48  def normalize_province_name(name: str) -> str:
49      result = str(name).strip()
50      replacements = {
51          " 特别行政区": "",
52          " 维吾尔自治区": "",

```

```

53     " 壮族自治区": "",
54     " 回族自治州": "",
55     " 自治区": "",
56     " 省": "",
57     " 市": "",
58 }
59 for old, new in replacements.items():
60     result = result.replace(old, new)
61 return result
62
63
64 def geometry_to_polygons(geometry: dict) -> list[np.ndarray]:
65     polygons = []
66     if geometry["type"] == "Polygon":
67         coordinates = [geometry["coordinates"]]
68     elif geometry["type"] == "MultiPolygon":
69         coordinates = geometry["coordinates"]
70     else:
71         return polygons
72     for polygon in coordinates:
73         if not polygon:
74             continue
75         exterior = np.asarray(polygon[0], dtype=float)
76         if len(exterior) >= 3:
77             polygons.append(exterior)
78     return polygons
79
80
81 def entropy_topsis(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
82     indicators = [
83         ("CO2 总量 _Mt", " 负向"),
84         (" 人均 CO2_ 吨每人", " 负向"),
85         (" 碳排放强度 _ 吨每万元 GDP", " 负向"),
86         (" 煤炭相关排放占比", " 负向"),
87         (" 第二产业占比", " 负向"),
88         (" 人均 GDP_ 万元每人", " 正向"),
89         (" 城镇化率 _%", " 正向"),
90     ]
91     matrix = []
92     for col, direction in indicators:
93         x = df[col].astype(float).to_numpy()
94         span = x.max() - x.min()
95         if span == 0:
96             z = np.ones_like(x)
97         elif direction == " 正向":
98             z = (x - x.min()) / span
99         else:
100             z = (x.max() - x) / span
101         matrix.append(z + 1e-12)
102
103     z = np.vstack(matrix).T
104     p = z / z.sum(axis=0)
105     entropy = -(1 / np.log(len(df))) * np.sum(p * np.log(p), axis=0)
106     weights = (1 - entropy) / (1 - entropy).sum()
107     d_pos = np.sqrt(((z - z.max(axis=0)) ** 2 * weights).sum(axis=1))
108     d_neg = np.sqrt(((z - z.min(axis=0)) ** 2 * weights).sum(axis=1))
109     score = d_neg / (d_pos + d_neg)
110
111     weights_df = pd.DataFrame(
112         {
113             " 指标": [col for col, _ in indicators],
114             " 属性": [direction for _, direction in indicators],
115             " 熵值": entropy,
116             " 权重": weights,
117         }
118     )
119     result = df[[" 省份"] + [col for col, _ in indicators]].copy()
120     result["TOPSIS 低碳得分"] = score
121     result["TOPSIS 排名"] = result["TOPSIS 低碳得分"].rank(ascending=False, method="min").astype(int)
122     result[" 低碳等级"] = pd.qcut(
123         result["TOPSIS 低碳得分"].rank(method="first"),
124         q=4,
125         labels=["IV 级 _ 转型压力大", "III 级 _ 中等压力", "II 级 _ 较好", "I 级 _ 优秀"],
126     )
127     return result.sort_values("TOPSIS 排名"), weights_df
128
129
130 def run_problem_one(province: pd.DataFrame) -> dict[str, pd.DataFrame]:
131     key_cols = [
132         "CO2 总量 _Mt",
133         " 人口 _ 万人",
134         "GDP_ 亿元",
135         " 人均 CO2_ 吨每人",
136         " 碳排放强度 _ 吨每万元 GDP",
137         " 煤炭相关排放占比",
138         " 第二产业占比",
139         " 人均 GDP_ 万元每人",
140         " 城镇化率 _%",
141     ]
142     require_columns(province, [" 省份"] + key_cols)
143     if len(province) != 30:
144         raise ValueError(f" 省级主表应为 30 行, 当前为 {len(province)} 行")

```

```

145
146 spatial_cols = ["CO2 总量 _Mt", " 人均 CO2_ 吨每人", " 碳排放强度 _ 吨每万元 GDP", " 煤炭相关排放占比"]
147 spatial_stats = []
148 for col in spatial_cols:
149     values = province[col].astype(float)
150     spatial_stats.append(
151         {
152             " 指标": col,
153             " 均值": values.mean(),
154             " 标准差": values.std(ddof=0),
155             " 变异系数": values.std(ddof=0) / values.mean(),
156             " 基尼系数": gini(values),
157             " 泰尔指数": theil(values),
158             " 最小值": values.min(),
159             " 最大值": values.max(),
160         }
161     )
162 spatial_stats_df = pd.DataFrame(spatial_stats)
163 toptsis_df, weights_df = entropy_topsis(province)
164
165 cluster_features = [
166     "CO2 总量 _Mt",
167     " 人均 CO2_ 吨每人",
168     " 碳排放强度 _ 吨每万元 GDP",
169     " 煤炭相关排放占比",
170     " 第二产业占比",
171     " 人均 GDP_ 万元每人",
172     " 城镇化率 _%",
173 ]
174 x = StandardScaler().fit_transform(province[cluster_features])
175 silhouette_df = pd.DataFrame(
176     [
177         {
178             "K": k,
179             " 轮廓系数": silhouette_score(x, KMeans(n_clusters=k, n_init=50, random_state=42).fit_predict(x)),
180         }
181         for k in range(2, 7)
182     ]
183 )
184
185 labels = KMeans(n_clusters=4, n_init=100, random_state=42).fit_predict(x)
186 cluster_df = province[[" 省份"] + cluster_features].copy()
187 cluster_df[" 聚类编号"] = labels
188 profile = cluster_df.groupby(" 聚类编号")[cluster_features].mean()
189 high_resource = profile[" 人均 CO2_ 吨每人"].idxmax()
190 efficient = profile[" 碳排放强度 _ 吨每万元 GDP"].idxmin()
191 remaining = [idx for idx in profile.index if idx not in {high_resource, efficient}]
192 industrial = profile.loc[remaining, "CO2 总量 _Mt"].idxmax()
193 transition = [idx for idx in remaining if idx != industrial][0]
194 cluster_names = {
195     high_resource: " 资源依赖高排放型",
196     industrial: " 工业制造高排放型",
197     transition: " 中等转型压力型",
198     efficient: " 经济发达效率型",
199 }
200 cluster_df[" 类型"] = cluster_df[" 聚类编号"].map(cluster_names)
201 cluster_profile = (
202     cluster_df.groupby(" 类型")[cluster_features]
203     .mean()
204     .reset_index()
205     .sort_values("CO2 总量 _Mt", ascending=False)
206 )
207
208 RESULT_DIR.mkdir(parents=True, exist_ok=True)
209 write_csv(spatial_stats_df, RESULT_DIR / "01_ 空间差异指标.csv")
210 write_csv(weights_df, RESULT_DIR / "02_ 熵权指标权重.csv")
211 write_csv(topsis_df, RESULT_DIR / "03_TOPSIS 得分排名.csv")
212 write_csv(silhouette_df, RESULT_DIR / "04_ 聚类 K 值检验.csv")
213 write_csv(cluster_df.sort_values([" 类型", " 省份"]), RESULT_DIR / "05_K4 聚类结果.csv")
214 write_csv(cluster_profile, RESULT_DIR / "06_ 类型均值画像.csv")
215
216 return {
217     "spatial_stats": spatial_stats_df,
218     "topsis": toptsis_df,
219     "weights": weights_df,
220     "silhouette": silhouette_df,
221     "cluster": cluster_df,
222     "cluster_profile": cluster_profile,
223 }
224
225
226 def make_problem_one_figures(problem_one: dict[str, pd.DataFrame]) -> None:
227     toptsis = problem_one["topsis"].copy()
228     cluster = problem_one["cluster"].copy()
229     province = toptsis.merge(cluster[[" 省份", " 类型"]], on=" 省份", how="left")
230
231     top_total = province.sort_values("CO2 总量 _Mt", ascending=True)
232     fig, ax = plt.subplots(figsize=(9, 8))
233     ax.barh(top_total[" 省份"], top_total["CO2 总量 _Mt"], color=sns.color_palette("viridis", n_colors=len(top_total)))
234     ax.set_title("2022 年 30 省 CO2 排放总量排名")
235     ax.set_xlabel("CO2 排放总量 (Mt)")
236     ax.set_ylabel("")

```

```

237 save_fig(fig, FIG_DIR, "01_ 省份 CO2 总量排名图")
238
239 fig, ax = plt.subplots(figsize=(9, 6))
240 scatter_data = province.rename(columns={"CO2 总量 _Mt": "CO2 总量 (Mt)"})
241 sns.scatterplot(
242     data=scatter_data,
243     x=" 人均 CO2_ 吨每人",
244     y=" 碳排放强度 _ 吨每万元 GDP",
245     hue=" 类型",
246     size="CO2 总量 (Mt)",
247     sizes=(120, 900),
248     alpha=0.8,
249     ax=ax,
250 )
251 label_offsets = {
252     " 内蒙古": (-54, -2),
253     " 宁夏": (18, 6),
254     " 山西": (26, -4),
255     " 新疆": (24, 8),
256     " 河北": (30, -8),
257 }
258 label_data = province[province[" 类型"].eq(" 资源依赖高排放型")]
259 for _, row in label_data.iterrows():
260     offset = label_offsets.get(row[" 省份"], (18, 6))
261     ax.annotate(
262         row[" 省份"],
263         xy=(row[" 人均 CO2_ 吨每人"], row[" 碳排放强度 _ 吨每万元 GDP"]),
264         xytext=offset,
265         textcoords="offset points",
266         ha="right" if offset[0] < 0 else "left",
267         va="center",
268         fontsize=9,
269         annotation_clip=False,
270     )
271 ax.set_title(" 人均 CO2 与碳排放强度关系")
272 ax.set_xlabel(" 人均 CO2 (吨/人) ")
273 ax.set_ylabel(" 碳排放强度 (吨/万元 GDP) ")
274 handles, labels = ax.get_legend_handles_labels()
275 ax.legend_.remove()
276 size_header = "CO2 总量 (Mt) "
277 size_start = labels.index(size_header)
278 type_handles = handles[1:size_start]
279 type_labels = labels[1:size_start]
280 size_handles = handles[size_start + 1 :]
281 size_labels = labels[size_start + 1 :]
282 type_legend = ax.legend(type_handles, type_labels, title=" 类型", loc="upper left", frameon=True)
283 ax.add_artist(type_legend)
284 ax.legend(size_handles, size_labels, title=size_header, loc="lower right", frameon=True)
285 save_fig(fig, FIG_DIR, "02_ 人均 CO2 与碳排放强度散点图")
286
287 if not CHINA_GEOJSON.exists():
288     raise FileNotFoundError(f" 缺少中国省级边界文件: {CHINA_GEOJSON}")
289 geojson = json.loads(CHINA_GEOJSON.read_text(encoding="utf-8"))
290 value_map = dict(zip(province[" 省份"].map(normalize_province_name), province["CO2 总量 _Mt"]))
291 values = np.array(list(value_map.values()), dtype=float)
292 norm = plt.Normalize(values.min(), values.max())
293 cmap = plt.get_cmap("YlOrRd")
294
295 fig, ax = plt.subplots(figsize=(14, 9.2))
296 missing_patches = []
297 data_patches = []
298 data_colors = []
299 label_points = []
300 callout_positions = {
301     " 北京": (124.4, 40.2),
302     " 天津": (124.4, 38.4),
303     " 上海": (124.2, 31.2),
304     " 海南": (113.3, 18.7),
305 }
306 text_effect = [path_effects.withStroke(linewidth=3.8, foreground="white")]
307 for feature in geojson["features"]:
308     raw_name = feature.get("properties", {}).get("name", "")
309     if not raw_name:
310         continue
311     province_name = normalize_province_name(raw_name)
312     polygons = geometry_to_polygons(feature["geometry"])
313     if province_name in value_map:
314         color = cmap(norm(value_map[province_name]))
315         for polygon in polygons:
316             data_patches.append(MplPolygon(polygon, closed=True))
317             data_colors.append(color)
318             center = feature.get("properties", {}).get("centroid") or feature.get("properties", {}).get("center")
319             if center:
320                 label_points.append((province_name, center[0], center[1], value_map[province_name]))
321     else:
322         for polygon in polygons:
323             missing_patches.append(MplPolygon(polygon, closed=True))
324
325 if missing_patches:
326     ax.add_collection(PatchCollection(missing_patches, facecolor="#E5E7EB", edgecolor="white", linewidth=0.7, zorder=1))
327 if data_patches:
328     ax.add_collection(PatchCollection(data_patches, facecolor=data_colors, edgecolor="white", linewidth=0.8, zorder=2))

```



```

329 for name, x_coord, y_coord, value in label_points:
330     label = f"{name}\n{value:.1f}"
331     if name in callout_positions:
332         text_x, text_y = callout_positions[name]
333         ax.plot([x_coord, text_x], [y_coord, text_y], color="#111827", linewidth=0.8, zorder=3)
334         ax.scatter([x_coord], [y_coord], s=10, color="#111827", zorder=4)
335         text = ax.text(
336             text_x,
337             text_y,
338             label,
339             fontsize=8.4,
340             ha="left",
341             va="center",
342             color="#111827",
343             linespacing=1.12,
344             zorder=5,
345         )
346     else:
347         text = ax.text(
348             x_coord,
349             y_coord,
350             label,
351             fontsize=8.2,
352             ha="center",
353             va="center",
354             color="#111827",
355             linespacing=1.12,
356             zorder=4,
357         )
358     text.set_path_effects(text_effect)
359 sm = plt.cm.ScalarMappable(norm=norm, cmap=cmap)
360 sm.set_array([])
361 cbar = fig.colorbar(sm, ax=ax, fraction=0.032, pad=0.02)
362 cbar.set_label("CO2 总量 (Mt) ")
363 ax.legend(handles=[Patch(facecolor="#E5E7EB", edgecolor="white", label="无数据或未纳入30省样本")], loc="lower left",
364           ↪ frameon=True)
365 ax.set_title("2022 年省级 CO2 排放总量空间分布")
366 ax.set_xlim(72, 136)
367 ax.set_ylim(17, 55)
368 ax.set_aspect("equal")
369 ax.axis("off")
370 save_fig(fig, FIG_DIR, "05_ 省级 CO2 总量空间分布热力图")
371
372 topsis_sorted = topsis.sort_values("TOPSIS 低碳得分", ascending=True)
373 fig, ax = plt.subplots(figsize=(9, 8))
374 ax.barh(topsis_sorted["省份"], topsis_sorted["TOPSIS 低碳得分"], color=sns.color_palette("crest", len(topsis_sorted)))
375 ax.set_title(" 熵权 TOPSIS 低碳发展得分排名")
376 ax.set_xlabel("TOPSIS 低碳得分")
377 ax.set_ylabel("")
378 save_fig(fig, FIG_DIR, "03_TOPSIS 低碳得分排名图")
379
380 old_radar = FIG_DIR / "04_ 聚类类型均值画像雷达图.png"
381 if old_radar.exists():
382     old_radar.unlink()
383
384 profile = problem_one["cluster_profile"].copy()
385 radar_cols = ["CO2总量_Mt", "人均CO2_吨每人", "碳排放强度_吨每万元GDP", "煤炭相关排放占比", "第二产业占比", "人均GDP_万元每人",
386 ↪ "城镇化率_%"]
387 scaled = profile.copy()
388 for col in radar_cols:
389     mn, mx = cluster[col].min(), cluster[col].max()
390     scaled[col] = (profile[col] - mn) / (mx - mn) if mx > mn else 0
391 labels = ["总量", "人均", "强度", "煤炭", "二产", "人均GDP", "城镇化"]
392 angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False)
393 polygon_angles = np.r_[angles, angles[0]]
394 palette = dict(zip(scaled["类型"], sns.color_palette("deep", n_colors=len(scaled))))
395
396 for idx, (_, row) in enumerate(scaled.iterrows(), start=1):
397     values = np.array([row[col] for col in radar_cols], dtype=float)
398     closed_values = np.r_[values, values[0]]
399     color = palette[row["类型"]]
400
401     fig = plt.figure(figsize=(7, 7))
402     ax = fig.add_subplot(111, polar=True)
403     ax.grid(False)
404     ax.spines["polar"].set_visible(False)
405     ax.set_ylim(0, 1)
406     ax.set_yticks([])
407     ax.set_theta_offset(np.pi / 2)
408     ax.set_theta_direction(-1)
409
410     # Draw polygon gridlines and radial spokes manually, replacing circular grids.
411     for radius in [0.2, 0.4, 0.6, 0.8, 1.0]:
412         ax.plot(polygon_angles, [radius] * len(polygon_angles), color="#D1D5DB", linewidth=1)
413     for angle in angles:
414         ax.plot([angle, angle], [0, 1], color="#D1D5DB", linewidth=1)
415
416     ax.plot(polygon_angles, closed_values, color=color, linewidth=2.6)
417     ax.fill(polygon_angles, closed_values, color=color, alpha=0.20)
418     ax.scatter(angles, values, color=color, s=42, zorder=3)
419     ax.set_xticks(angles)
420     ax.set_xticklabels(labels, fontsize=11)

```

```

419         ax.set_title(row[" 类型"], fontsize=17, pad=22)
420         save_fig(fig, FIG_DIR, f"04_{idx}_{row['类型']}_ 均值画像雷达图")
421
422
423     def main() -> None:
424         setup_style()
425         province, _, _ = read_data()
426         result = run_problem_one(province)
427         make_problem_one_figures(result)
428         print(f" 问题一结果已生成: {RESULT_DIR}")
429         print(f" 问题一图片已生成: {FIG_DIR}")
430
431
432     if __name__ == "__main__":
433         main()

```

1.4 问题二求解脚本 solve_problem2.py

```

1  from __future__ import annotations
2
3  """
4  问题二脚本说明
5
6  算法与模型:
7  - STIRPAT 横截面回归:  $\ln(CO2)=a+b1 \ln(P)+b2 \ln(A)+b3 \text{ Coal}+b4 \text{ IS}+b5 \text{ U}+e$ 。
8  - OLS 显著性检验: 输出系数、标准误、t 值、p 值、R2、调整 R2。
9  - VIF 共线性检验: 判断解释变量是否存在严重多重共线性。
10 - 岭回归留一交叉验证: 用正则化模型检验驱动因素排序和预测稳健性。
11
12 使用数据:
13 - 输入: 数据/2022 省级综合表.xlsx, 工作表" 主表"。
14 - 输出: 1. 结果/问题二/ 下多个 CSV 结果文件。
15 - 输出图片: 2. 图片/问题二/05_STIRPAT 标准化系数图、06_OLS 拟合值与真实值对比图。
16 """
17
18 import sys
19 from pathlib import Path
20
21 CODE_ROOT = Path(__file__).resolve().parents[1]
22 ROOT = CODE_ROOT.parent
23 sys.pycache_prefix = str(ROOT / ".pycache")
24 sys.path.append(str(CODE_ROOT))
25
26 import matplotlib.pyplot as plt
27 from matplotlib.lines import Line2D
28 import numpy as np
29 import pandas as pd
30 import seaborn as sns
31 import statsmodels.api as sm
32 from sklearn.linear_model import RidgeCV
33 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
34 from sklearn.model_selection import LeaveOneOut, cross_val_predict
35 from sklearn.preprocessing import StandardScaler
36 from statsmodels.stats.outliers_influence import variance_inflation_factor
37
38 from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, read_data, save_fig, setup_style, write_csv
39
40
41 PROBLEM = " 问题二"
42 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
43 FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
44
45
46 def run_problem_two(province: pd.DataFrame) -> dict[str, pd.DataFrame]:
47     df = province.copy()
48     df["lnC"] = np.log(df["CO2 总量_Mt"])
49     df["lnP"] = np.log(df[" 人口 _ 万人"])
50     df["lnA"] = np.log(df[" 人均 GDP_ 万元每人"])
51     df[" 城镇化率"] = df[" 城镇化率_%"] / 100
52     predictors = ["lnP", "lnA", " 煤炭相关排放占比", " 第二产业占比", " 城镇化率"]
53     x = sm.add_constant(df[predictors])
54     y = df["lnC"]
55     ols = sm.OLS(y, x).fit()
56
57     ols_df = pd.DataFrame(
58         {
59             " 变量": ols.params.index,
60             " 系数": ols.params.values,
61             " 标准误": ols.bse.values,
62             "t 值": ols.tvalues.values,
63             "p 值": ols.pvalues.values,
64         }
65     )
66     fit_df = pd.DataFrame(
67         {
68             " 指标": ["R2", " 调整 R2", "F 统计量", "F 检验 p 值", "AIC", "BIC", "RMSE_log", "MAE_log"],
69             " 数值": [
70                 ols.rsquared,
71                 ols.rsquared_adj,
72                 ols.fvalue,

```

```

73         ols.f_pvalue,
74         ols.aic,
75         ols.bic,
76         mean_squared_error(y, ols.fittedvalues) ** 0.5,
77         mean_absolute_error(y, ols.fittedvalues),
78     ],
79     }
80 )
81 vif_df = pd.DataFrame(
82     {
83         "变量": predictors,
84         "VIF": [variance_inflation_factor(x.values, i + 1) for i in range(len(predictors))],
85     }
86 )
87 fitted_df = pd.DataFrame(
88     {
89         "省份": df["省份"],
90         "实际 lnCO2": y,
91         "拟合 lnCO2": ols.fittedvalues,
92         "残差": ols.resid,
93         "实际 CO2_Mt": df["CO2 总量_Mt"],
94         "拟合 CO2_Mt": np.exp(ols.fittedvalues),
95     }
96 )
97 fitted_df["绝对误差_Mt"] = (fitted_df["拟合 CO2_Mt"] - fitted_df["实际 CO2_Mt"]).abs()
98 fitted_df["误差方向"] = np.where(fitted_df["拟合 CO2_Mt"] > fitted_df["实际 CO2_Mt"], "模型高估", "模型低估")
99
100 scaler = StandardScaler()
101 xs = scaler.fit_transform(df[predictors])
102 ridge_cv = RidgeCV(alphas=np.logspace(-3, 3, 100), cv=LeaveOneOut())
103 ridge_pred = cross_val_predict(ridge_cv, xs, y, cv=LeaveOneOut())
104 ridge_cv.fit(xs, y)
105 ridge_coef_df = pd.DataFrame(
106     {
107         "变量": predictors,
108         "标准化岭回归系数": ridge_cv.coef_,
109         "绝对值排序": np.abs(ridge_cv.coef_).argsort()[::-1].argsort() + 1,
110     }
111 ).sort_values("绝对值排序")
112 ridge_metrics_df = pd.DataFrame(
113     {
114         "指标": ["最优 alpha", "LOOCV_R2", "LOOCV_RMSE_log", "LOOCV_MAE_log"],
115         "数值": [
116             ridge_cv.alpha_,
117             r2_score(y, ridge_pred),
118             mean_squared_error(y, ridge_pred) ** 0.5,
119             mean_absolute_error(y, ridge_pred),
120         ],
121     }
122 )
123
124 RESULT_DIR.mkdir(parents=True, exist_ok=True)
125 write_csv(ols_df, RESULT_DIR / "01_OLS 系数.csv")
126 write_csv(fit_df, RESULT_DIR / "02_拟合优度.csv")
127 write_csv(vif_df, RESULT_DIR / "03_VIF 共线性检验.csv")
128 write_csv(fitted_df, RESULT_DIR / "04_拟合值残差.csv")
129 write_csv(ridge_metrics_df, RESULT_DIR / "05_岭回归交叉验证.csv")
130 write_csv(ridge_coef_df, RESULT_DIR / "06_岭回归标准化系数.csv")
131
132 return {
133     "ols_coef": ols_df,
134     "fit": fit_df,
135     "vif": vif_df,
136     "fitted": fitted_df,
137     "ridge_metrics": ridge_metrics_df,
138     "ridge_coef": ridge_coef_df,
139 }
140
141
142 def make_problem_two_figures(problem_two: dict[str, pd.DataFrame]) -> None:
143     ridge_coef = problem_two["ridge_coef"].sort_values("标准化岭回归系数")
144     variable_labels = {
145         "lnP": "人口规模 (lnP)",
146         "lnA": "经济发展水平 (lnA)",
147         "煤炭相关排放占比": "煤炭相关排放占比",
148         "第二产业占比": "第二产业占比",
149         "城镇化率": "城镇化率",
150     }
151     variable_colors = {
152         "lnP": "#2563EB",
153         "lnA": "#059669",
154         "煤炭相关排放占比": "#DC2626",
155         "第二产业占比": "#D97706",
156         "城镇化率": "#7C3AED",
157     }
158     ridge_coef["变量说明"] = ridge_coef["变量"].map(variable_labels).fillna(ridge_coef["变量"])
159     bar_colors = ridge_coef["变量"].map(variable_colors).fillna("#4B5563")
160     fig, ax = plt.subplots(figsize=(8, 5))
161     ax.barh(ridge_coef["变量说明"], ridge_coef["标准化岭回归系数"], color=bar_colors)
162     ax.axvline(0, color="#333333", linewidth=1)
163     ax.set_title("STIRPAT 岭回归标准化系数")
164     ax.set_xlabel("标准化系数")

```

```

165 ax.set_ylabel("")
166 save_fig(fig, FIG_DIR, "05_STIRPAT 标准化系数图")
167
168 fitted = problem_two["fitted"].copy()
169 fitted[" 误差 _Mt"] = fitted[" 拟合 CO2_Mt"] - fitted[" 实际 CO2_Mt"]
170 lim_min = min(fitted[" 实际 CO2_Mt"].min(), fitted[" 拟合 CO2_Mt"].min()) * 0.88
171 lim_max = max(fitted[" 实际 CO2_Mt"].max(), fitted[" 拟合 CO2_Mt"].max()) * 1.08
172 plot_data = fitted.rename(
173     columns={
174         " 实际 CO2_Mt": " 实际 CO2 排放量 (Mt) ",
175         " 拟合 CO2_Mt": " 拟合 CO2 排放量 (Mt) ",
176         " 绝对误差 _Mt": " 绝对误差 (Mt) ",
177     }
178 )
179 plot_data[" 误差方向"] = plot_data[" 误差方向"].map({" 模型高估": " 高估", " 模型低估": " 低估"}).fillna(plot_data[" 误差方向"])
180 error_palette = {" 高估": "#C2410C", " 低估": "#2563EB"}
181
182 fig, ax = plt.subplots(figsize=(8.5, 7))
183 sns.scatterplot(
184     data=plot_data,
185     x=" 实际 CO2 排放量 (Mt) ",
186     y=" 拟合 CO2 排放量 (Mt) ",
187     hue=" 误差方向",
188     size=" 绝对误差 (Mt) ",
189     sizes=(50, 280),
190     palette=error_palette,
191     alpha=0.82,
192     legend=False,
193     ax=ax,
194 )
195 ax.plot([lim_min, lim_max], [lim_min, lim_max], "--", color="#333333", linewidth=1.5)
196 direction_handles = [
197     Line2D([0], [0], marker="o", linestyle="", color=color, label=label, markersize=8, alpha=0.82)
198     for label, color in error_palette.items()
199 ]
200 direction_legend = ax.legend(direction_handles, [" 高估", " 低估"], title=" 误差方向", loc="upper left", frameon=True)
201 ax.add_artist(direction_legend)
202 size_values = [150, 300, 450, 600]
203 size_handles = [
204     Line2D([0], [0], marker="o", linestyle="", color="#595959", label=str(value), markersize=np.sqrt(value / 600) * 16)
205     for value in size_values
206 ]
207 line_handle = Line2D([0], [0], linestyle="--", color="#333333", linewidth=1.5, label=" 完全拟合线")
208 ax.legend(
209     size_handles + [line_handle],
210     [str(value) for value in size_values] + [" 完全拟合线"],
211     title=" 绝对误差 (Mt) ",
212     loc="lower right",
213     frameon=True,
214 )
215 ax.set_xlim(lim_min, lim_max)
216 ax.set_ylim(lim_min, lim_max)
217 ax.set_title("STIRPAT 模型实际值与拟合值对比及误差说明")
218 ax.set_xlabel(" 实际 CO2 排放量 (Mt) ")
219 ax.set_ylabel(" 模型拟合 CO2 排放量 (Mt) ")
220 save_fig(fig, FIG_DIR, "06_OLS 拟合值与真实值对比图")
221
222
223 def main() -> None:
224     setup_style()
225     province, _, _ = read_data()
226     result = run_problem_two(province)
227     make_problem_two_figures(result)
228     print(f" 问题二结果已生成: {RESULT_DIR}")
229     print(f" 问题二图片已生成: {FIG_DIR}")
230
231
232 if __name__ == "__main__":
233     main()

```

1.5 问题三求解脚本 solve_problem3.py

```

1 from __future__ import annotations
2
3 """
4 问题三脚本说明
5
6 算法与模型:
7 - 全国时间趋势统计: 只使用附件 1 中 Sector=Total 的全国排放总量, 避免 Total 与部门排放重复相加。
8 - STIRPAT 系数递推预测: 使用问题二 OLS 系数, 按驱动变量变化推算全国排放相对变化。
9 - 达峰研判: 逐情景寻找预测序列最大值, 输出达峰年份、峰值水平、碳排放强度和减排潜力。
10
11 使用数据:
12 - 输入: 数据/附件 1: 2019-2025 年全国碳排放数据.csv、数据/全国数据.xlsx、问题二 STIRPAT 系数。
13 - 输出: 1. 结果/问题三/ 下多个 CSV 结果文件。
14 - 输出图片: 2. 图片/问题三/07_ 三情景碳排放趋势图、08_ 三情景峰值与减排潜力对比图。
15 """
16
17 import sys
18 from pathlib import Path

```

```

19
20 CODE_ROOT = Path(__file__).resolve().parents[1]
21 ROOT = CODE_ROOT.parent
22 sys.pycache_prefix = str(ROOT / ".pycache")
23 sys.path.append(str(CODE_ROOT))
24
25 import matplotlib.pyplot as plt
26 import numpy as np
27 import pandas as pd
28 import seaborn as sns
29
30 from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, read_data, save_fig, scenario_driver_assumptions, setup_style,
↪ write_csv
31 from 问题二.solve_problem2 import run_problem_two
32
33
34 PROBLEM = " 问题三"
35 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
36 FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
37
38
39 def _annual_rate(start: float, end: float, year: int, first_year: int = 2025, last_year: int = 2045) -> float:
40     if last_year == first_year:
41         return start
42     ratio = (year - first_year) / (last_year - first_year)
43     return start + (end - start) * ratio
44
45
46 def _prepare_national_base(national: pd.DataFrame, province: pd.DataFrame, base_year: int) -> tuple[dict[str, float],
↪ pd.DataFrame]:
47     national_std = national.rename(
48         columns={
49             " 年 份": " 年份",
50             "GDP (亿元)": "GDP_ 亿元",
51             " 人口 (万人)": " 人口 _ 万人",
52             " 能源消费总量 (万吨标准煤)": " 能源消费总量 _ 万吨标准煤",
53             " 煤炭占比": " 煤炭占比 _%",
54         }
55     ).copy()
56     national_std[" 人均 GDP_ 万元每人"] = national_std["GDP_ 亿元"] / national_std[" 人口 _ 万人"]
57     national_std[" 煤炭占比"] = national_std[" 煤炭占比 _%" ] / 100
58     base = national_std[national_std[" 年份"].eq(base_year)]
59     if base.empty:
60         raise ValueError(f" 全国数据缺少 {base_year} 年, 无法确定问题三预测基准。")
61     base_row = base.iloc[0]
62     industry_base = province[" 第二产业 _ 亿元"].sum() / province["GDP_ 亿元"].sum()
63     urban_base = (province[" 城镇化率 _%"] * province[" 人口 _ 万人"]).sum() / province[" 人口 _ 万人"].sum() / 100
64     base_values = {
65         " 年份": base_year,
66         " 人口 _ 万人": float(base_row[" 人口 _ 万人"]),
67         "GDP_ 亿元": float(base_row["GDP_ 亿元"]),
68         " 人均 GDP_ 万元每人": float(base_row[" 人均 GDP_ 万元每人"]),
69         " 煤炭相关排放占比": float(base_row[" 煤炭占比"]),
70         " 第二产业占比": float(industry_base),
71         " 城镇化率": float(urban_base),
72     }
73     base_table = pd.DataFrame(
74         [
75             {" 变量": " 人口 _ 万人", " 基准值": base_values[" 人口 _ 万人"], " 来源": " 全国数据.xlsx 2024 年人口"},
76             {" 变量": "GDP_ 亿元", " 基准值": base_values["GDP_ 亿元"], " 来源": " 全国数据.xlsx 2024 年 GDP"},
77             {" 变量": " 人均 GDP_ 万元每人", " 基准值": base_values[" 人均 GDP_ 万元每人"], " 来源": "GDP/人口"},
78             {" 变量": " 煤炭相关排放占比", " 基准值": base_values[" 煤炭相关排放占比"], " 来源": " 全国数据.xlsx 2024 年煤炭占比"},
79             {" 变量": " 第二产业占比", " 基准值": base_values[" 第二产业占比"], " 来源": "2022 省级综合表按 GDP 聚合"},
80             {" 变量": " 城镇化率", " 基准值": base_values[" 城镇化率"], " 来源": "2022 省级综合表按人口加权"},
81         ]
82     )
83     return base_values, base_table
84
85
86 def _simulate_forecast(
87     base_emission: float,
88     base_values: dict[str, float],
89     coeff: dict[str, float],
90     assumptions: dict[str, dict[str, float]],
91     base_year: int = 2024,
92 ) -> tuple[pd.DataFrame, pd.DataFrame]:
93     rows = []
94     param_rows = []
95     for scenario, params in assumptions.items():
96         emission = base_emission
97         population = base_values[" 人口 _ 万人"]
98         affluence = base_values[" 人均 GDP_ 万元每人"]
99         coal_share = base_values[" 煤炭相关排放占比"]
100         industry_share = base_values[" 第二产业占比"]
101         urban_rate = base_values[" 城镇化率"]
102         gdp = population * affluence
103         rows.append(
104             {
105                 " 情景": scenario,
106                 " 年份": base_year,
107                 " 预测 CO2-Mt": emission,
108                 " 预测 GDP_ 亿元": gdp,

```

```

109         "碳排放强度_吨每万元GDP": emission / gdp * 100,
110         "人口_万人": population,
111         "人均GDP_万元每人": affluence,
112         "煤炭相关排放占比": coal_share,
113         "第二产业占比": industry_share,
114         "城镇化率": urban_rate,
115         "STIRPAT 对数增长项": 0.0,
116     }
117 )
118 for year in range(2025, 2046):
119     g_population = _annual_rate(params["人口增长率_起始"], params["人口增长率_末期"], year)
120     g_affluence = _annual_rate(params["人均GDP增长率_起始"], params["人均GDP增长率_末期"], year)
121     d_coal = _annual_rate(params["煤炭占比年变化_起始"], params["煤炭占比年变化_末期"], year)
122     d_industry = _annual_rate(params["第二产业占比年变化_起始"], params["第二产业占比年变化_末期"], year)
123     d_urban = _annual_rate(params["城镇化率年变化_起始"], params["城镇化率年变化_末期"], year)
124     prev_population = population
125     prev_affluence = affluence
126     prev_coal = coal_share
127     prev_industry = industry_share
128     prev_urban = urban_rate
129     population *= 1 + g_population
130     affluence *= 1 + g_affluence
131     coal_share = float(np.clip(coal_share + d_coal, 0.05, 0.95))
132     industry_share = float(np.clip(industry_share + d_industry, 0.20, 0.60))
133     urban_rate = float(np.clip(urban_rate + d_urban, 0.45, 0.90))
134     d_ln_population = np.log(population / prev_population)
135     d_ln_affluence = np.log(affluence / prev_affluence)
136     d_coal_actual = coal_share - prev_coal
137     d_industry_actual = industry_share - prev_industry
138     d_urban_actual = urban_rate - prev_urban
139     log_growth = (
140         coeff["lnP"] * d_ln_population
141         + coeff["lnA"] * d_ln_affluence
142         + coeff["煤炭相关排放占比"] * d_coal_actual
143         + coeff["第二产业占比"] * d_industry_actual
144         + coeff["城镇化率"] * d_urban_actual
145     )
146     emission *= float(np.exp(log_growth))
147     gdp = population * affluence
148     param_rows.append(
149         {
150             "情景": scenario,
151             "年份": year,
152             "人口增长率": g_population,
153             "人均GDP增长率": g_affluence,
154             "煤炭占比年变化": d_coal_actual,
155             "第二产业占比年变化": d_industry_actual,
156             "城镇化率年变化": d_urban_actual,
157             "DeltalnP": d_ln_population,
158             "DeltalnA": d_ln_affluence,
159             "STIRPAT 对数增长项": log_growth,
160             "排放增长率": np.exp(log_growth) - 1,
161         }
162     )
163     rows.append(
164         {
165             "情景": scenario,
166             "年份": year,
167             "预测 CO2_Mt": emission,
168             "预测 GDP_亿元": gdp,
169             "碳排放强度_吨每万元GDP": emission / gdp * 100,
170             "人口_万人": population,
171             "人均GDP_万元每人": affluence,
172             "煤炭相关排放占比": coal_share,
173             "第二产业占比": industry_share,
174             "城镇化率": urban_rate,
175             "STIRPAT 对数增长项": log_growth,
176         }
177     )
178 return pd.DataFrame(rows), pd.DataFrame(param_rows)
179
180
181 def _summarize_peak(forecast_df: pd.DataFrame, scenario_order: list[str], base_year: int = 2024) -> pd.DataFrame:
182     peak_rows = []
183     for scenario, group in forecast_df.groupby("情景"):
184         group = group.sort_values("年份")
185         idx = group["预测 CO2_Mt"].idxmax()
186         peak_year = int(group.loc[idx, "年份"])
187         peak_value = float(group.loc[idx, "预测 CO2_Mt"])
188         base_intensity = float(group.loc[group["年份"].eq(base_year), "碳排放强度_吨每万元GDP"].iloc[0])
189         intensity_2045 = float(group.loc[group["年份"].eq(2045), "碳排放强度_吨每万元GDP"].iloc[0])
190         peak_rows.append(
191             {
192                 "情景": scenario,
193                 "达峰年份": peak_year,
194                 "峰值_Mt": peak_value,
195                 "2030年_Mt": float(group.loc[group["年份"].eq(2030), "预测 CO2_Mt"].iloc[0]),
196                 "2035年_Mt": float(group.loc[group["年份"].eq(2035), "预测 CO2_Mt"].iloc[0]),
197                 "2045年_Mt": float(group.loc[group["年份"].eq(2045), "预测 CO2_Mt"].iloc[0]),
198                 "2045较峰值下降比例": 1 - float(group.loc[group["年份"].eq(2045), "预测 CO2_Mt"].iloc[0]) / peak_value,
199                 "2030年强度_吨每万元GDP": float(group.loc[group["年份"].eq(2030), "碳排放强度_吨每万元GDP"].iloc[0]),
200                 "2045年强度_吨每万元GDP": intensity_2045,

```

```

201         "2045 较 2024 强度下降比例": 1 - intensity_2045 / base_intensity,
202     }
203 )
204 peak_df = pd.DataFrame(peak_rows)
205 peak_df["情景"] = pd.Categorical(peak_df["情景"], categories=scenario_order, ordered=True)
206 peak_df = peak_df.sort_values("情景").reset_index(drop=True)
207 peak_df["情景"] = peak_df["情景"].astype(str)
208 return peak_df
209
210
211 def _copy_assumptions(assumptions: dict[str, dict[str, float]]) -> dict[str, dict[str, float]]:
212     return {scenario: values.copy() for scenario, values in assumptions.items()}
213
214
215 def _single_factor_sensitivity(
216     base_emission: float,
217     base_values: dict[str, float],
218     coeff: dict[str, float],
219     assumptions: dict[str, dict[str, float]],
220     baseline_peak: pd.DataFrame,
221 ) -> pd.DataFrame:
222     variable_keys = {
223         "人口增长率": ["人口增长率 _ 起始", "人口增长率 _ 末期"],
224         "人均 GDP 增长率": ["人均 GDP 增长率 _ 起始", "人均 GDP 增长率 _ 末期"],
225         "煤炭占比下降速度": ["煤炭占比年变化 _ 起始", "煤炭占比年变化 _ 末期"],
226         "第二产业占比下降速度": ["第二产业占比年变化 _ 起始", "第二产业占比年变化 _ 末期"],
227         "城镇化率年变化": ["城镇化率年变化 _ 起始", "城镇化率年变化 _ 末期"],
228     }
229     perturbations = [-0.20, -0.10, 0.10, 0.20]
230     baseline = baseline_peak.set_index("情景")
231     rows = []
232     scenario_order = list(assumptions.keys())
233     for scenario in scenario_order:
234         for variable, keys in variable_keys.items():
235             for perturbation in perturbations:
236                 perturbed = _copy_assumptions(assumptions)
237                 for key in keys:
238                     perturbed[scenario][key] *= 1 + perturbation
239                 forecast_df, _ = _simulate_forecast(base_emission, base_values, coeff, perturbed)
240                 peak = _summarize_peak(forecast_df, scenario_order)
241                 result = peak.set_index("情景").loc[scenario]
242                 base = baseline.loc[scenario]
243                 rows.append(
244                     {
245                         "情景": scenario,
246                         "扰动变量": variable,
247                         "扰动比例": perturbation,
248                         "达峰年份": result["达峰年份"],
249                         "达峰年份变化": result["达峰年份"] - base["达峰年份"],
250                         "峰值_Mt": result["峰值_Mt"],
251                         "峰值变化_Mt": result["峰值_Mt"] - base["峰值_Mt"],
252                         "2045 年_Mt": result["2045 年_Mt"],
253                         "2045 变化_Mt": result["2045 年_Mt"] - base["2045 年_Mt"],
254                         "2045 较峰值下降比例": result["2045 较峰值下降比例"],
255                     }
256                 )
257     return pd.DataFrame(rows)
258
259
260 def _coefficient_sensitivity(
261     base_emission: float,
262     base_values: dict[str, float],
263     coeff: dict[str, float],
264     assumptions: dict[str, dict[str, float]],
265     baseline_peak: pd.DataFrame,
266 ) -> pd.DataFrame:
267     variable_labels = {
268         "lnP": "人口规模系数",
269         "lnA": "人均 GDP 系数",
270         "煤炭相关排放占比": "煤炭占比系数",
271         "第二产业占比": "第二产业占比系数",
272         "城镇化率": "城镇化率系数",
273     }
274     perturbations = [-0.10, 0.10]
275     baseline = baseline_peak.set_index("情景")
276     rows = []
277     scenario_order = list(assumptions.keys())
278     for coeff_name, label in variable_labels.items():
279         for perturbation in perturbations:
280             perturbed_coeff = coeff.copy()
281             perturbed_coeff[coeff_name] *= 1 + perturbation
282             forecast_df, _ = _simulate_forecast(base_emission, base_values, perturbed_coeff, assumptions)
283             peak = _summarize_peak(forecast_df, scenario_order)
284             for scenario in scenario_order:
285                 result = peak.set_index("情景").loc[scenario]
286                 base = baseline.loc[scenario]
287                 rows.append(
288                     {
289                         "情景": scenario,
290                         "扰动系数": coeff_name,
291                         "系数说明": label,
292                         "扰动比例": perturbation,

```

```

293         "达峰年份": result["达峰年份"],
294         "达峰年份变化": result["达峰年份"] - base["达峰年份"],
295         "峰值_Mt": result["峰值_Mt"],
296         "峰值变化_Mt": result["峰值_Mt"] - base["峰值_Mt"],
297         "2045年_Mt": result["2045年_Mt"],
298         "2045变化_Mt": result["2045年_Mt"] - base["2045年_Mt"],
299         "2045较峰值下降比例": result["2045较峰值下降比例"],
300     }
301 )
302 return pd.DataFrame(rows)
303
304
305 def run_problem_three(
306     carbon: pd.DataFrame,
307     national: pd.DataFrame,
308     province: pd.DataFrame,
309     problem_two: dict[str, pd.DataFrame] | None = None,
310 ) -> dict[str, pd.DataFrame]:
311     total = carbon[carbon["Sector"].eq("Total")].copy()
312     annual = total.groupby("Year", as_index=False)["CO2 (Mt)"].sum()
313     days = total.groupby("Year")["Date"].nunique().reset_index(name="记录天数")
314     annual = annual.merge(days, on="Year", how="left")
315     annual["是否完整年份"] = annual["记录天数"] >= 365
316     base_year = 2024
317     if base_year not in annual["Year"].values:
318         raise ValueError("附件1缺少2024年Sector=Total数据,无法确定预测基准。")
319     base_emission = float(annual.loc[annual["Year"].eq(base_year), "CO2 (Mt)"].iloc[0])
320     base_values, base_table = _prepare_national_base(national, province, base_year)
321     if problem_two is None:
322         problem_two = run_problem_two(province)
323     coeff = problem_two["ols_coef"].set_index("变量")["系数"].to_dict()
324     required = ["lnP", "lnA", "煤炭相关排放占比", "第二产业占比", "城镇化率"]
325     missing = [name for name in required if name not in coeff]
326     if missing:
327         raise ValueError(f"问题二 OLS 系数缺少变量: {missing}")
328
329     coeff_rows = pd.DataFrame(
330         [{"变量": key, "STIRPAT_OLS 系数": coeff[key], "预测中含义": "驱动变量变化对 ln(CO2) 的边际影响"} for key in required]
331     )
332     assumptions = scenario_driver_assumptions()
333     scenario_order = list(assumptions.keys())
334     forecast_df, params_df = _simulate_forecast(base_emission, base_values, coeff, assumptions, base_year)
335     peak_df = _summarize_peak(forecast_df, scenario_order, base_year)
336     single_factor_sensitivity = _single_factor_sensitivity(base_emission, base_values, coeff, assumptions, peak_df)
337     coefficient_sensitivity = _coefficient_sensitivity(base_emission, base_values, coeff, assumptions, peak_df)
338     sector_2024 = (
339         carbon[(carbon["Year"].eq(2024)) & (~carbon["Sector"].eq("Total"))]
340         .groupby("Sector", as_index=False)["CO2 (Mt)"]
341         .sum()
342         .sort_values("CO2 (Mt)", ascending=False)
343     )
344
345     RESULT_DIR.mkdir(parents=True, exist_ok=True)
346     write_csv(annual, RESULT_DIR / "01_全国年度排放.csv")
347     write_csv(sector_2024, RESULT_DIR / "02_2024 部门排放.csv")
348     write_csv(params_df, RESULT_DIR / "03_情景参数.csv")
349     write_csv(forecast_df, RESULT_DIR / "04_2024-2045 预测序列.csv")
350     write_csv(peak_df, RESULT_DIR / "05_达峰与减排潜力.csv")
351     write_csv(coeff_rows, RESULT_DIR / "06_STIRPAT 递推系数.csv")
352     write_csv(base_table, RESULT_DIR / "07_驱动变量基准值.csv")
353     write_csv(single_factor_sensitivity, RESULT_DIR / "08_单因素灵敏度分析.csv")
354     write_csv(coefficient_sensitivity, RESULT_DIR / "09_STIRPAT 系数灵敏度分析.csv")
355
356     return {
357         "annual": annual,
358         "sector_2024": sector_2024,
359         "params": params_df,
360         "forecast": forecast_df,
361         "peak": peak_df,
362         "stirpat_coef": coeff_rows,
363         "driver_base": base_table,
364         "single_factor_sensitivity": single_factor_sensitivity,
365         "coefficient_sensitivity": coefficient_sensitivity,
366     }
367
368
369 def make_problem_three_figures(problem_three: dict[str, pd.DataFrame]) -> None:
370     forecast = problem_three["forecast"]
371     peak = problem_three["peak"].copy()
372     scenario_order = ["基准情景", "低碳情景", "强化低碳情景"]
373     fig, ax = plt.subplots(figsize=(13, 5.5))
374     plot_data = forecast[forecast["年份"].between(2024, 2045)].rename(columns={"预测 CO2_Mt": "预测 CO2 排放量 (Mt)"})
375     sns.lineplot(
376         data=plot_data,
377         x="年份",
378         y="预测 CO2 排放量 (Mt)",
379         hue="情景",
380         hue_order=scenario_order,
381         linewidth=2.4,
382         marker="o",
383         markersize=3,
384         ax=ax,

```



```

385 )
386 color_map = {text.get_text(): handle.get_color() for text, handle in zip(ax.legend_.texts, ax.legend_.legend_handles)}
387 y_min = plot_data[" 预测 CO2 排放量 (Mt) "].min() * 0.97
388 ax.set_ylim(y_min, plot_data[" 预测 CO2 排放量 (Mt) "].max() * 1.03)
389 peak[" 情景"] = pd.Categorical(peak[" 情景"], categories=scenario_order, ordered=True)
390 peak = peak.sort_values(" 情景").reset_index(drop=True)
391 peak[" 情景"] = peak[" 情景"].astype(str)
392 for _, row in peak.iterrows():
393     color = color_map.get(row[" 情景"], "#555555")
394     ax.vlines(row[" 达峰年份"], y_min, row[" 峰值 _Mt"], color=color, linestyle="--", linewidth=1.2, alpha=0.75)
395     ax.scatter(row[" 达峰年份"], row[" 峰值 _Mt"], color=color, edgecolor="white", linewidth=0.8, s=58, zorder=5)
396 peak_year_colors = {int(row[" 达峰年份"]): color_map.get(row[" 情景"], "#555555") for _, row in peak.iterrows()}
397 ax.set_xticks([2024, 2025, 2026, 2030, 2032, 2035, 2040, 2045])
398 ax.tick_params(axis="x", labelsize=10, pad=6)
399 for label in ax.get_xticklabels():
400     year = int(float(label.get_text()))
401     if year in peak_year_colors:
402         label.set_color(peak_year_colors[year])
403 ax.legend(title=" 情景", loc="upper left", bbox_to_anchor=(1.01, 1), frameon=True)
404 ax.set_title("2024-2045 年全国 CO2 排放三情景 STIRPAT 预测")
405 ax.set_xlabel(" 年份")
406 ax.set_ylabel("CO2 排放量 (Mt) ")
407 save_fig(fig, FIG_DIR, "07_ 三情景碳排放趋势图")
408
409 peak = problem_three["peak"].copy()
410 peak[" 情景"] = pd.Categorical(peak[" 情景"], categories=scenario_order, ordered=True)
411 peak = peak.sort_values(" 情景").reset_index(drop=True)
412 peak[" 情景"] = peak[" 情景"].astype(str)
413 fig, ax1 = plt.subplots(figsize=(8.5, 5.5))
414 scenario_palette = {" 基准情景": "#4C72B0", " 低碳情景": "#DD8452", " 强化低碳情景": "#55A868"}
415 sns.barplot(data=peak, x="情景", y="峰值_Mt", hue="情景", order=scenario_order, ax=ax1, palette=scenario_palette,
↳ legend=False)
416 for patch in ax1.patches:
417     current_width = patch.get_width()
418     new_width = current_width * 0.72
419     patch.set_x(patch.get_x() + (current_width - new_width) / 2)
420     patch.set_width(new_width)
421 ax1.set_title(" 三情景峰值水平与 2045 减排潜力")
422 ax1.set_ylabel(" 峰值排放 (Mt) ")
423 ax1.set_xlabel("")
424 left_ticks = [10000, 10600, 11200, 11800, 12400, 13000]
425 right_ticks = [0, 9, 18, 27, 36, 45]
426 ax1.set_ylim(10000, 13000)
427 ax1.set_yticks(left_ticks)
428 ax1.grid(True, axis="y")
429 ax2 = ax1.twinx()
430 line_color = "#4F46E5"
431 ax2.set_ylim(0, 45)
432 ax2.set_yticks(right_ticks)
433 ax2.grid(False)
434 ax2.plot(peak[" 情景"], peak["2045 较峰值下降比例"] * 100, color=line_color, marker="o", linewidth=2.2)
435 ax2.set_ylabel("2045 较峰值下降比例 (%) ")
436 for i, row in peak.reset_index().iterrows():
437     ax1.text(i, row[" 峰值 _Mt"], f'{row[" 达峰年份"]}', ha="center", va="bottom", fontsize=10)
438     ratio = row["2045 较峰值下降比例"] * 100
439     offset = 1.8 if i != 2 else -2.2
440     ax2.text(
441         i,
442         ratio + offset,
443         f'{row["2045 较峰值下降比例"]:.1%}',
444         ha="center",
445         va="bottom" if offset > 0 else "top",
446         color="#312E81",
447         fontsize=10,
448         bbox={"boxstyle": "round,pad=0.2", "facecolor": "white", "edgecolor": "none", "alpha": 0.82},
449     )
450 save_fig(fig, FIG_DIR, "08_ 三情景峰值与减排潜力对比图")
451
452 single = problem_three["single_factor_sensitivity"].copy()
453 single_plot = single[(single[" 情景"].eq(" 基准情景") & (single[" 扰动比例"].isin([-0.20, 0.20]))].copy()
454 single_plot[" 扰动方向"] = single_plot[" 扰动比例"].map({-0.20: "-20%", 0.20: "+20%"})
455 order = (
456     single_plot.groupby(" 扰动变量")["2045 变化 _Mt"]
457     .apply(lambda s: s.abs().max())
458     .sort_values(ascending=True)
459     .index.tolist()
460 )
461 fig, ax = plt.subplots(figsize=(8.8, 5.4))
462 sns.barplot(
463     data=single_plot,
464     y=" 扰动变量",
465     x="2045 变化 _Mt",
466     hue=" 扰动方向",
467     order=order,
468     palette={"-20%": "#60A5FA", "+20%": "#F97316"},
469     ax=ax,
470 )
471 ax.axvline(0, color="#333333", linewidth=1)
472 ax.set_title(" 单因素灵敏度: 基准情景 2045 排放影响")
473 ax.set_xlabel("2045 年 CO2 排放变化量 (Mt) ")
474 ax.set_ylabel("")
475 ax.legend(title=" 扰动比例", frameon=True)

```

```

476 save_fig(fig, FIG_DIR, "09_ 单因素灵敏度 _2045 排放影响图")
477
478 coef = problem_three["coefficient_sensitivity"].copy()
479 coef_plot = coef[(coef["情景"].eq("基准情景")) & (coef["扰动比例"].isin([-0.10, 0.10]))].copy()
480 coef_plot["扰动方向"] = coef_plot["扰动比例"].map({-0.10: "-10%", 0.10: "+10%"})
481 order = (
482     coef_plot.groupby("系数说明")["2045 变化 _Mt"]
483     .apply(lambda s: s.abs().max())
484     .sort_values(ascending=True)
485     .index.tolist()
486 )
487 fig, ax = plt.subplots(figsize=(8.8, 5.4))
488 sns.barplot(
489     data=coef_plot,
490     y="系数说明",
491     x="2045 变化 _Mt",
492     hue="扰动方向",
493     order=order,
494     palette={"-10%": "#93C5FD", "+10%": "#8B5CF6"},
495     ax=ax,
496 )
497 ax.axvline(0, color="#333333", linewidth=1)
498 ax.set_title("STIRPAT 系数灵敏度: 基准情景 2045 排放影响")
499 ax.set_xlabel("2045 年 CO2 排放变化量 (Mt)")
500 ax.set_ylabel("")
501 ax.legend(title="扰动比例", frameon=True)
502 save_fig(fig, FIG_DIR, "10_STIRPAT 系数灵敏度 _2045 排放影响图")
503
504
505 def main() -> None:
506     setup_style()
507     province, national, carbon = read_data()
508     problem_two = run_problem_two(province)
509     result = run_problem_three(carbon, national, province, problem_two)
510     make_problem_three_figures(result)
511     print(f"问题三结果已生成: {RESULT_DIR}")
512     print(f"问题三图片已生成: {FIG_DIR}")
513
514
515 if __name__ == "__main__":
516     main()

```

1.6 问题四求解脚本 solve_problem4.py

```

1 from __future__ import annotations
2
3 """
4 问题四脚本说明
5
6 算法与模型:
7 - 不再重新建模, 而是读取问题一的省份分类、问题二的驱动因素检验、问题三的情景预测结果。
8 - 按"区域分类-关键驱动-达峰预测"逻辑生成政策建议报告。
9
10 使用数据:
11 - 输入: 问题一、问题二、问题三脚本返回的结果字典, 或由 run_all.py 统一传入。
12 - 输出: 1. 结果/问题四/问题四_政策建议报告.md。
13 """
14
15 import sys
16 from pathlib import Path
17
18 CODE_ROOT = Path(__file__).resolve().parents[1]
19 ROOT = CODE_ROOT.parent
20 sys.pycache_prefix = str(ROOT / ".pycache")
21 sys.path.append(str(CODE_ROOT))
22
23 import pandas as pd
24
25 from common import PROBLEM_RESULT_DIRS, read_data, setup_style
26 from 问题一.solve_problem1 import run_problem_one
27 from 问题二.solve_problem2 import run_problem_two
28 from 问题三.solve_problem3 import run_problem_three
29
30
31 PROBLEM = "问题四"
32 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
33
34
35 def write_policy_report(problem_one: dict[str, pd.DataFrame], problem_two: dict[str, pd.DataFrame], problem_three: dict[str,
↵ pd.DataFrame]) -> str:
36     cluster = problem_one["cluster"]
37     ridge = problem_two["ridge_coef"].sort_values("绝对值排序")
38     peak = problem_three["peak"].set_index("情景")
39     resource = cluster[cluster["类型"].eq("资源依赖高排放型")]["省份"].tolist()
40     industrial = cluster[cluster["类型"].eq("工业制造高排放型")]["省份"].tolist()
41     main_drivers = ", ".join(ridge.head(2)["变量"].tolist())
42
43     policy = f"""# 问题四: 政策建议报告
44

```

```

45 基于省际分类、STIRPAT驱动因素识别和三情景预测结果，我国推进"双碳"目标应坚持"总量控制、结构优化、效率提升、区域分类"的思路。首先，对{"、
↳ ".join(resource)}等资源依赖高排放型省份，应将煤炭消费总量控制和煤电低碳改造作为重点，推动煤炭清洁高效利用、煤电灵活性改造、
↳ 新能源基地建设和资源型产业转型，避免经济增长继续依赖高碳能源扩张。其次，对{"、".join(industrial)}等工业制造高排放型省份，应聚焦钢铁、
↳ 建材、电力、石化和交通等重点行业，实施节能改造、余热利用、电气化替代和绿色供应链管理，建立单位产品碳强度约束。再次，
↳ 中等转型压力型省份应在承接产业转移时设置能耗和碳排放准入门槛，防止高耗能产业简单转移，同时提高能源利用效率和非化石能源占比。北京、
↳ 上海等经济发达效率型地区应发挥技术、金融和治理优势，推广绿色金融、碳交易、数字化碳管理和低碳消费。模型结果表明，
↳ {main_drivers}是最稳定的关键驱动因素：情景预测显示，基准情景约在{int(peak.loc["基准情景", "达峰年份"])}年达峰，
↳ 强化低碳情景下2045年较峰值下降{peak.loc["强化低碳情景", "2045较峰值下降比例"]:.1%}。因此，应持续降低煤炭占比，
↳ 扩大风光水核等非化石能源供给，强化技术创新和碳市场机制，形成差异化、可考核、可持续的减排路径。

46 """
47     RESULT_DIR.mkdir(parents=True, exist_ok=True)
48     (RESULT_DIR / " 问题四 _ 政策建议报告.md").write_text(policy, encoding="utf-8")
49     return policy
50
51
52 def main() -> None:
53     setup_style()
54     province, national, carbon = read_data()
55     problem_one = run_problem_one(province)
56     problem_two = run_problem_two(province)
57     problem_three = run_problem_three(carbon, national, province, problem_two)
58     write_policy_report(problem_one, problem_two, problem_three)
59     print(f" 问题四报告已生成: {RESULT_DIR}")
60
61
62 if __name__ == "__main__":
63     main()

```